



Universidade Federal de Santa Catarina – UFSC
DEC – Engenharia de Computação

Linguagem de Descrição de Hardware

Dr. Ney Laert Vilar Calazans

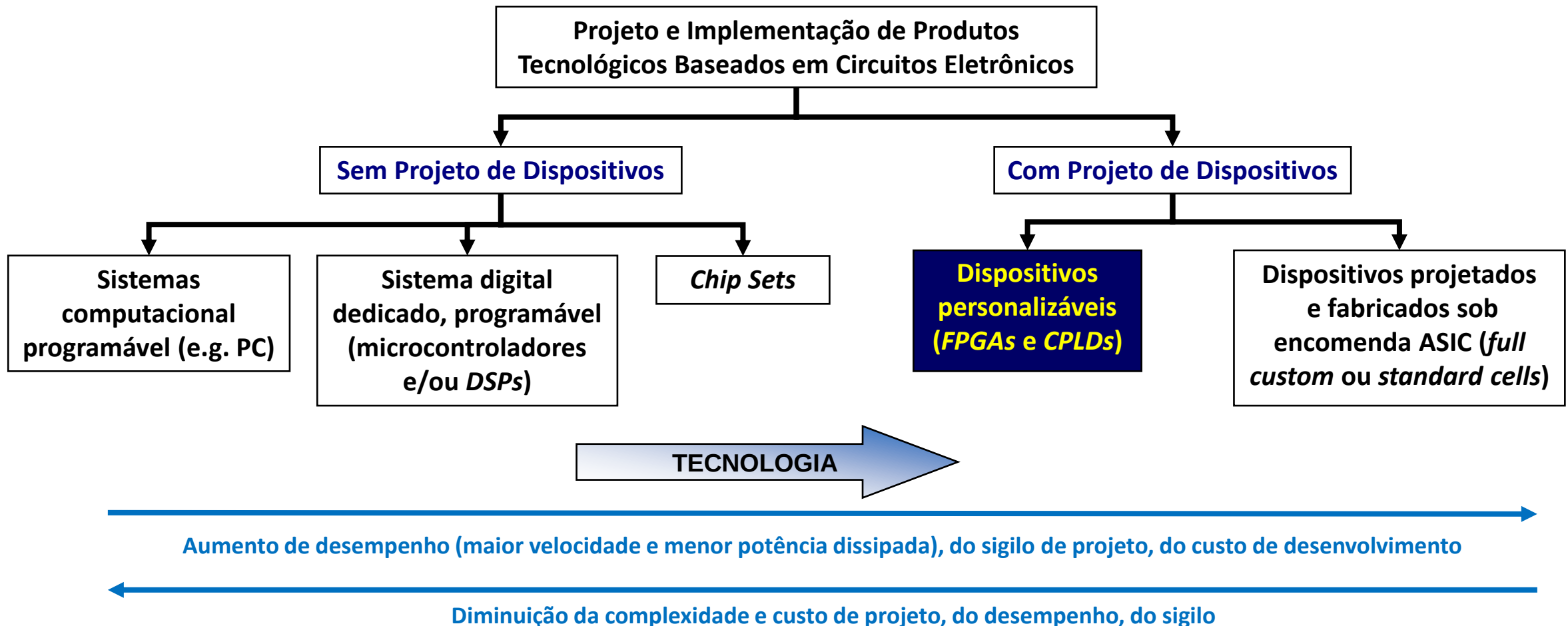
Última alteração: 02/06/2025

❖ Baseado em materiais originais dos Profs. **Ney Calazans**, **Fernando Moraes (PUCRS)**, e **Rafael Garibotti (PUCRS)**

TEORIA, HISTÓRICO E ESTRUTURA DE FPGAs

O QUE SÃO FPGAs?

- FPGAs permitem implementar sistemas digitais diretamente de HDLs, sem os custos de fabricação de chips!

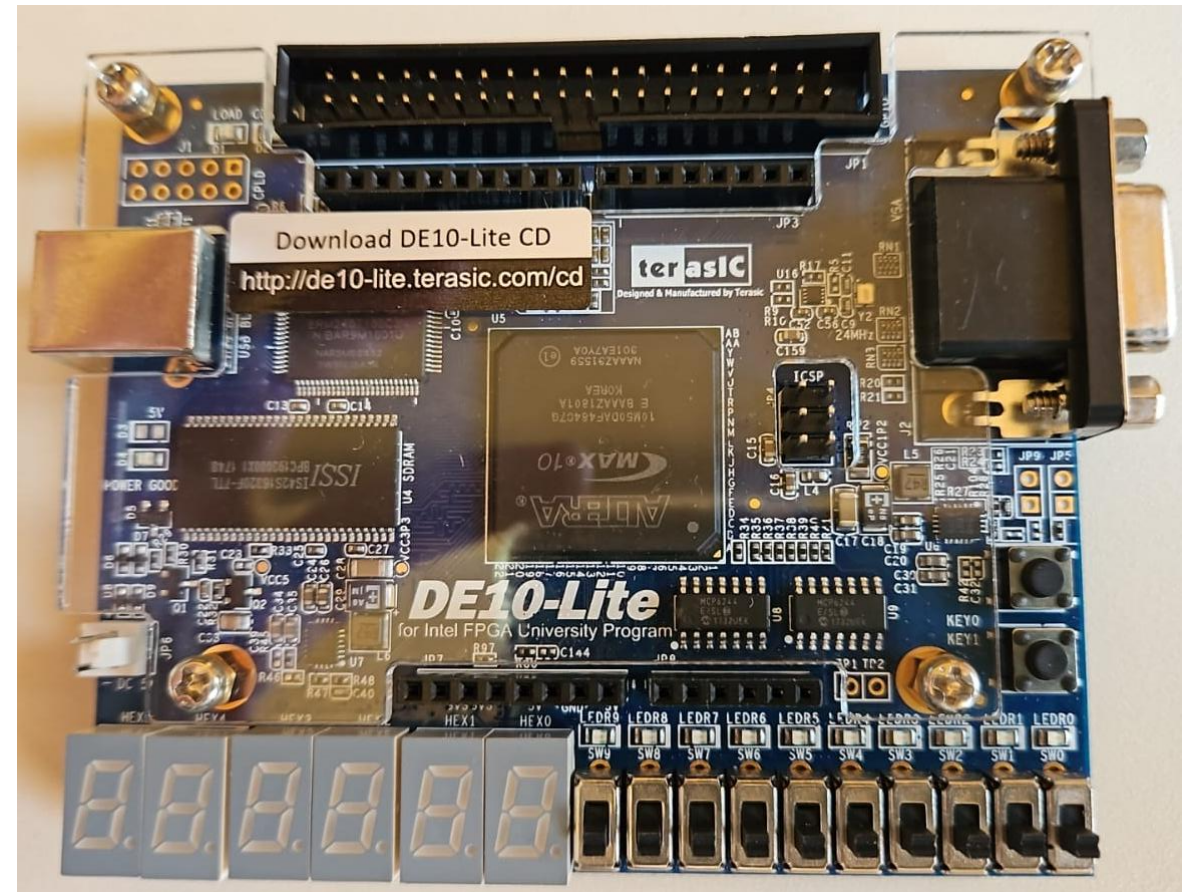


DOIS EXEMPLOS DE PLATAFORMAS BASEADAS EM FPGAS

Digilent Nexys 2



Terasic DE10-Lite

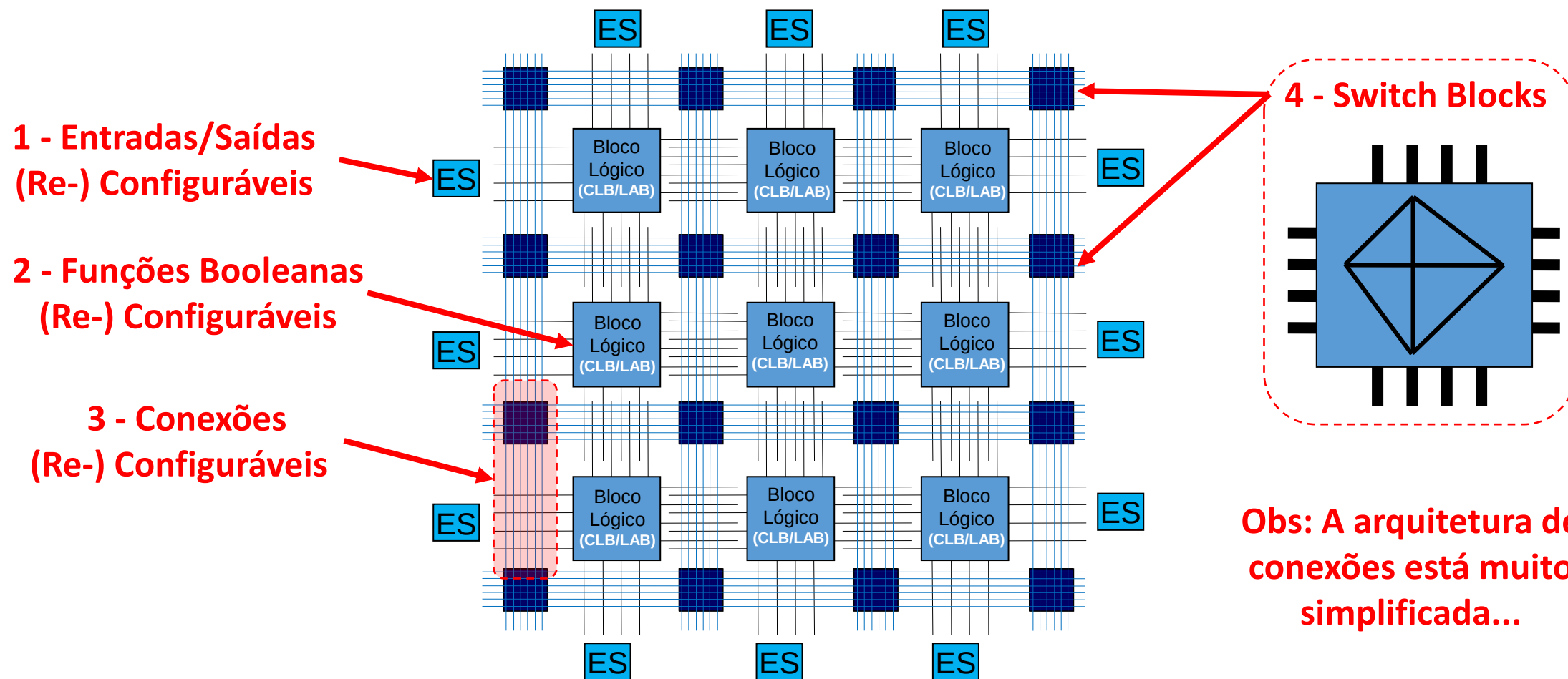


UM POUCO DA HISTÓRIA DE FPGAS

- Primeiro, vieram PROMs (*Programmable Read Only Memories*) e *PLDs* (*Programmable Logic Devices*), matrizes de portas lógicas (re-) configuráveis
- Algumas patentes de coisas parecidas com *FPGAs* (*Field Programmable Gate Arrays*) surgiram no final dos anos 80 e início dos anos 90 (Casselman, Page, Peterson)
- Os fundadores da [Xilinx](#), Ross Freeman e Bernard Vonderschmitt, inventaram o primeiro FPGA comercial em 1985 – o XC2064
- O XC2064 tinha 64 blocos lógicos configuráveis e interconexões configuráveis entre os blocos lógicos
- O XC2064 só tinha blocos lógicos configuráveis (CLBs), cada um com duas LUTs de 3 entradas

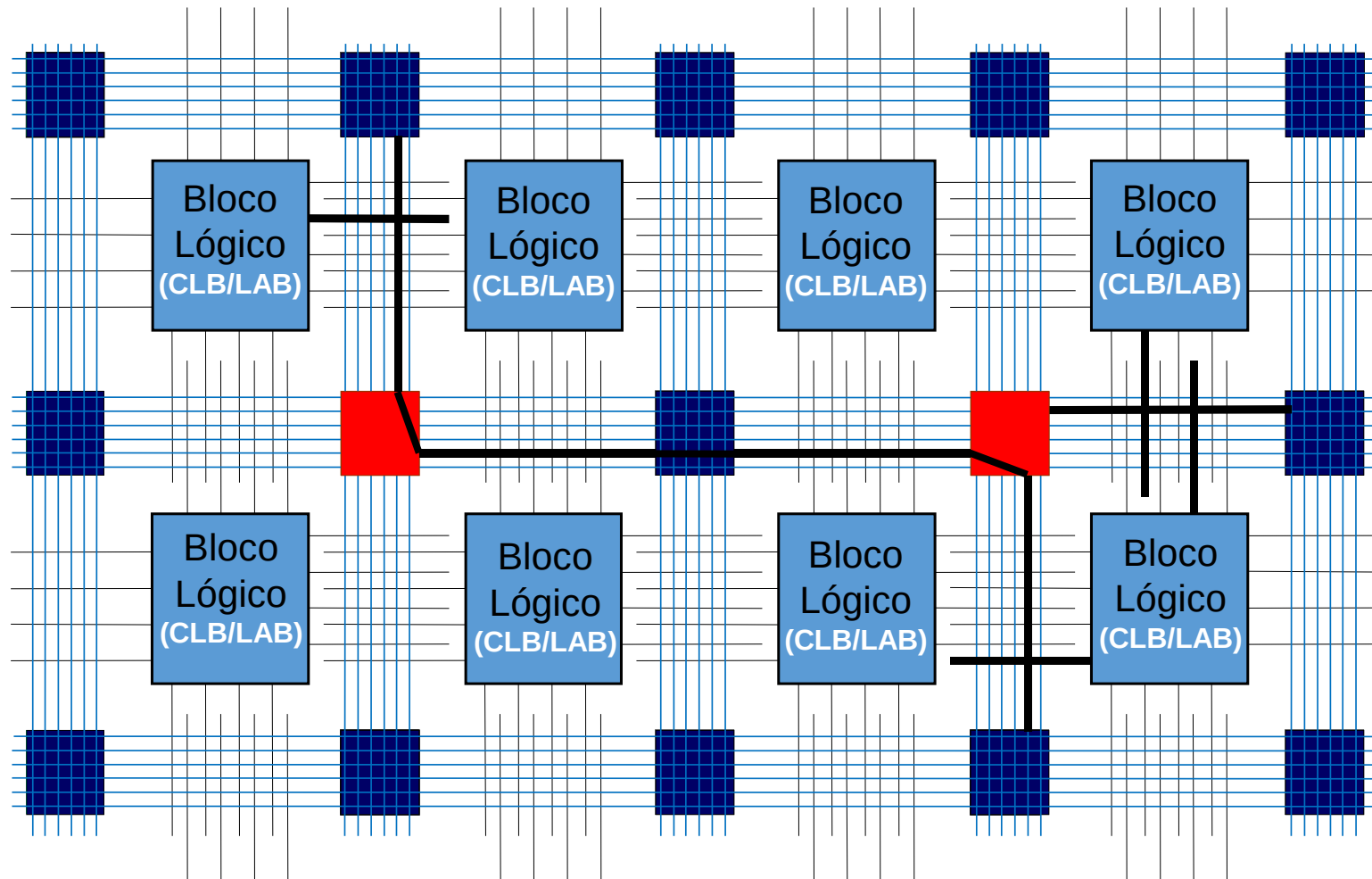
FPGAs – CONCEITOS BÁSICOS

- Matriz de *configurable logic blocks* (CLBs, nomenclatura [AMD/Xilinx](#)) ou *logic array blocks* (LABs, nomenclatura [Intel/Altera](#)), interconectados por matrizes de chaveamento



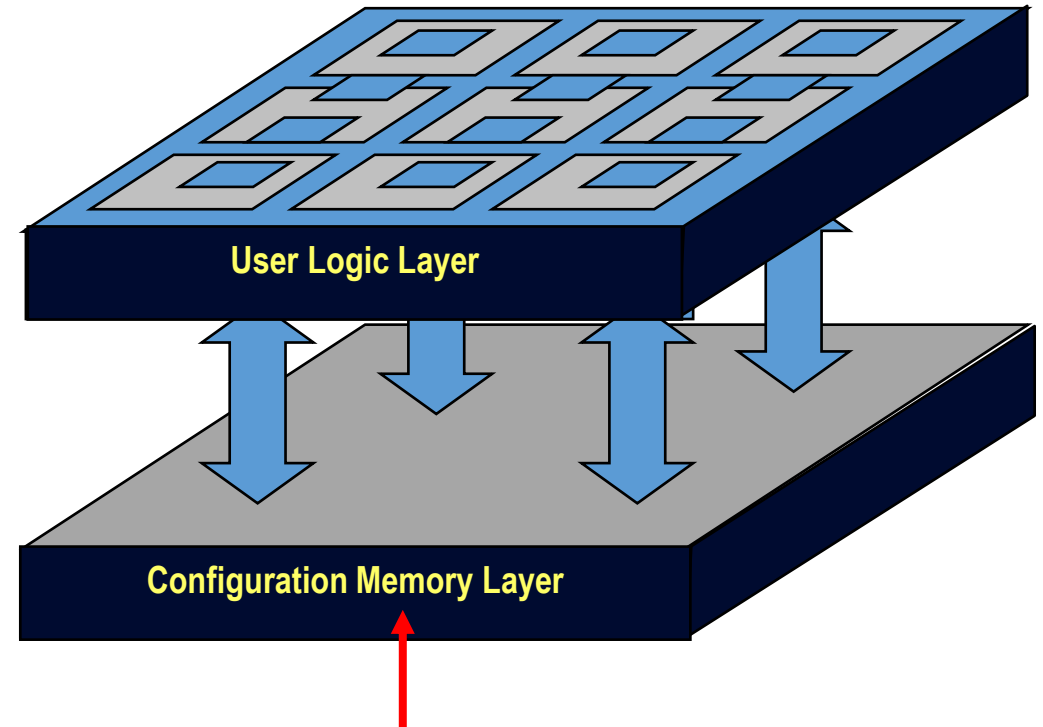
CONEXÕES ENTRE CLBS ou LABS

➤ Exemplo de conexão entre duas redes



FPGAs – CONFIGURAÇÃO (RAM-BASED)

- Um FPGA deve ser visto como um dispositivo de “duas camadas”
 1. Bloco 1 - Uma memória de configuração (90% do Hw do chip) associada a toda uma lógica que define o Bloco 2
 2. Bloco 2 - a lógica do usuário (10% do Hw)
- A memória de configuração define
 - ✓ Toda a fiação da lógica do usuário
 - ✓ As funções lógicas das LUTs (a ver...)
 - ✓ Interface externas e internas
 - ✓ Estrutura e conteúdo das memórias (ROM/RAM) do usuário
 - ✓ Configuração dos pinos de E/S



Por exemplo, na família Virtex 4, a memória de configuração ocupa algo entre 1 MByte – 4 MBytes

TECNOLOGIAS DE CONFIGURAÇÃO

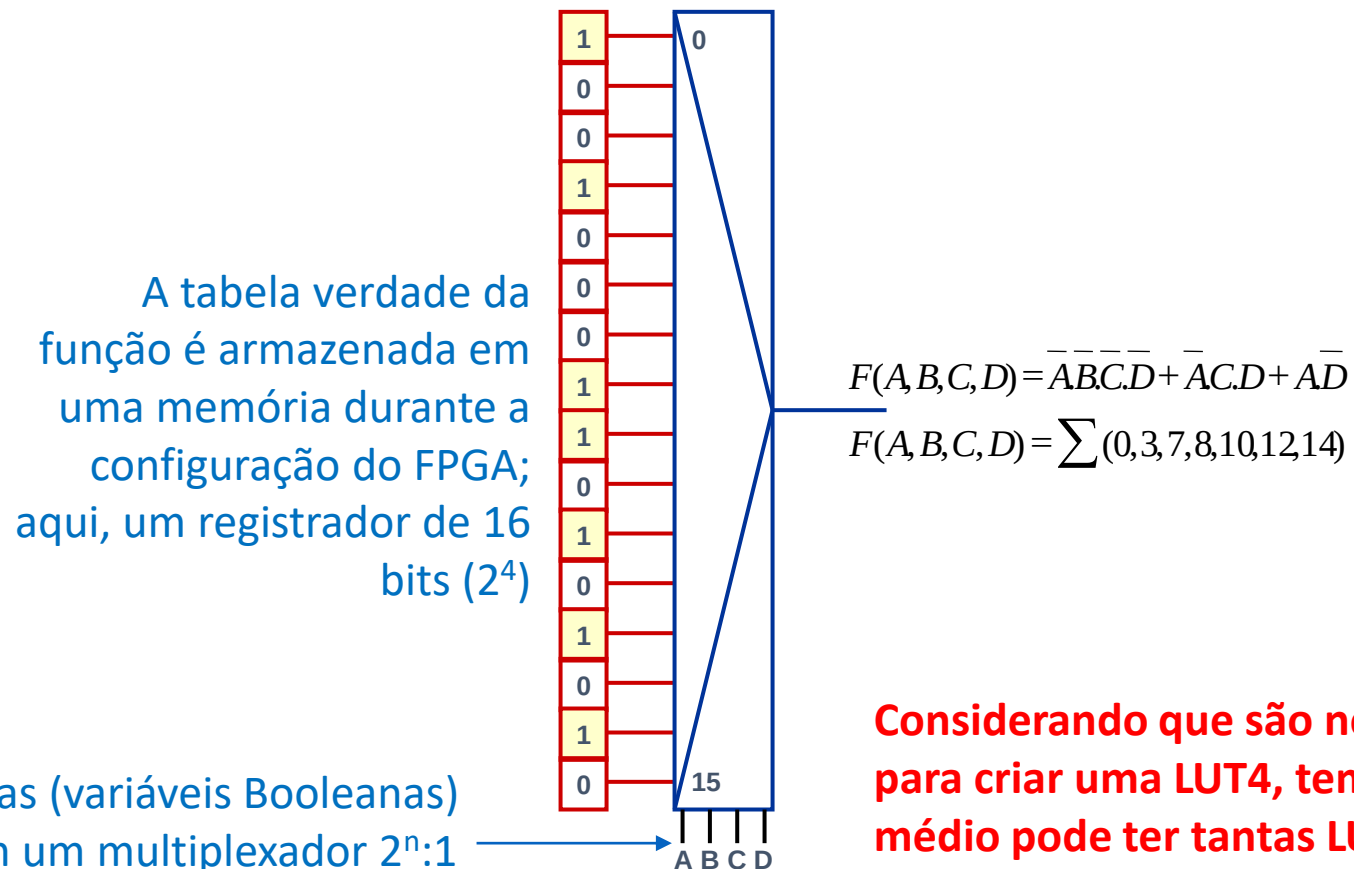
- Algumas das diferentes tecnologias usadas para definir o comportamento de um FPGA
 - ✓ **Antifusível**: Configuração uma única vez, bem barato, mas depois de configurado, fica assim “Para todo o Sempre, Amém!”
 - ✓ **(E)EPROM**: Configuração um número limitado de vezes (100 a 1000 vezes), mantida mesmo com o *chip* desconectado da alimentação
 - ✓ **Flash**: Tecnologia mais avançada que (E)EPROM, de mesma funcionalidade, mas mais durável e mais simples de configurar – **Placa DE10-Lite usa MAX10, flash-based!**
 - ✓ **SRAM**: Configuração deve ser realizada cada vez que o FPGA for alimentado, seja de forma manual, seja de forma automática (desligou, sumiu o Hw...) – **Placas Digilent Nexys e Nexys2 usam FPGAs baseados em SRAM (das famílias Spartan3 e Spartan3E, resp.)**

LUT – UM GERADOR UNIVERSAL DE FUNÇÕES

- **LUT** (*Look-Up Table*) – Um exemplo de bloco com função Booleana reconfigurável
 - ✓ Uma porção de hardware configurável/reconfigurável capaz de implementar **qualquer** tabela verdade de n entradas
 - ✓ Para $n=4$, existem 65.536 funções implementáveis distintas (2^{2^4}) – Mas por quê?
 - Altamente flexível
 - Método mais utilizado (Xilinx [AMD], Altera [Intel] e outros fabricantes usam LUTs)

LUT – UM GERADOR UNIVERSAL DE FUNÇÕES

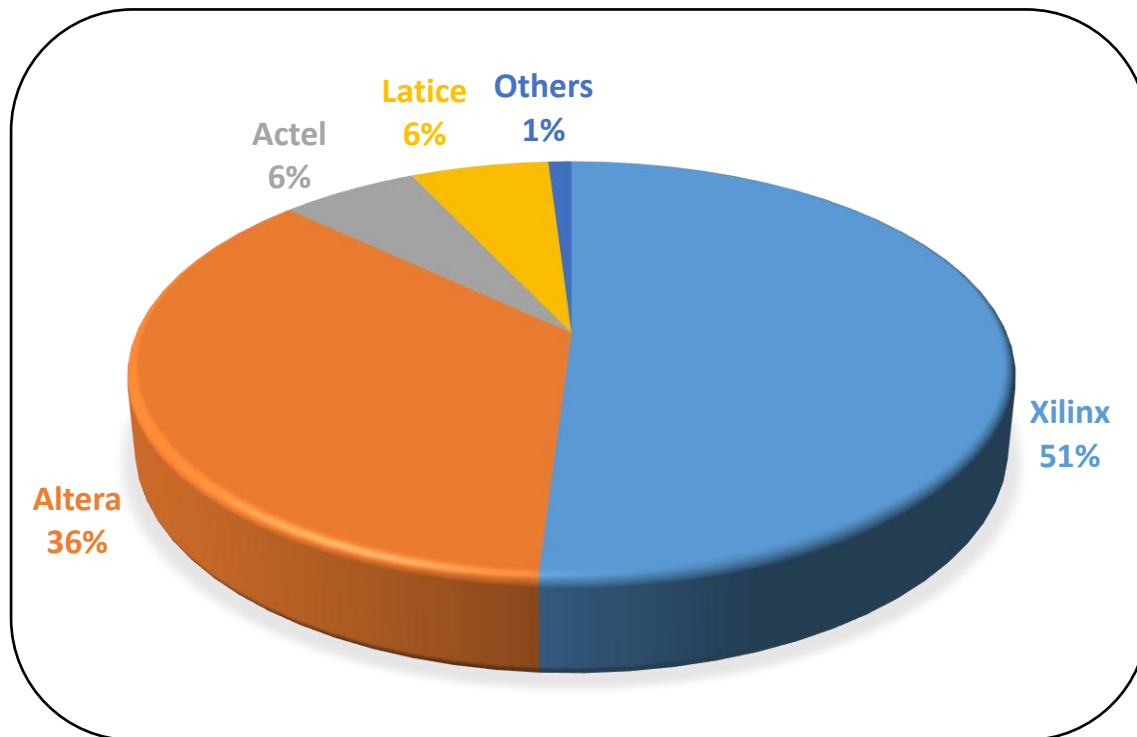
➤ Implementação física de uma LUT4 → notem como é simples!!!



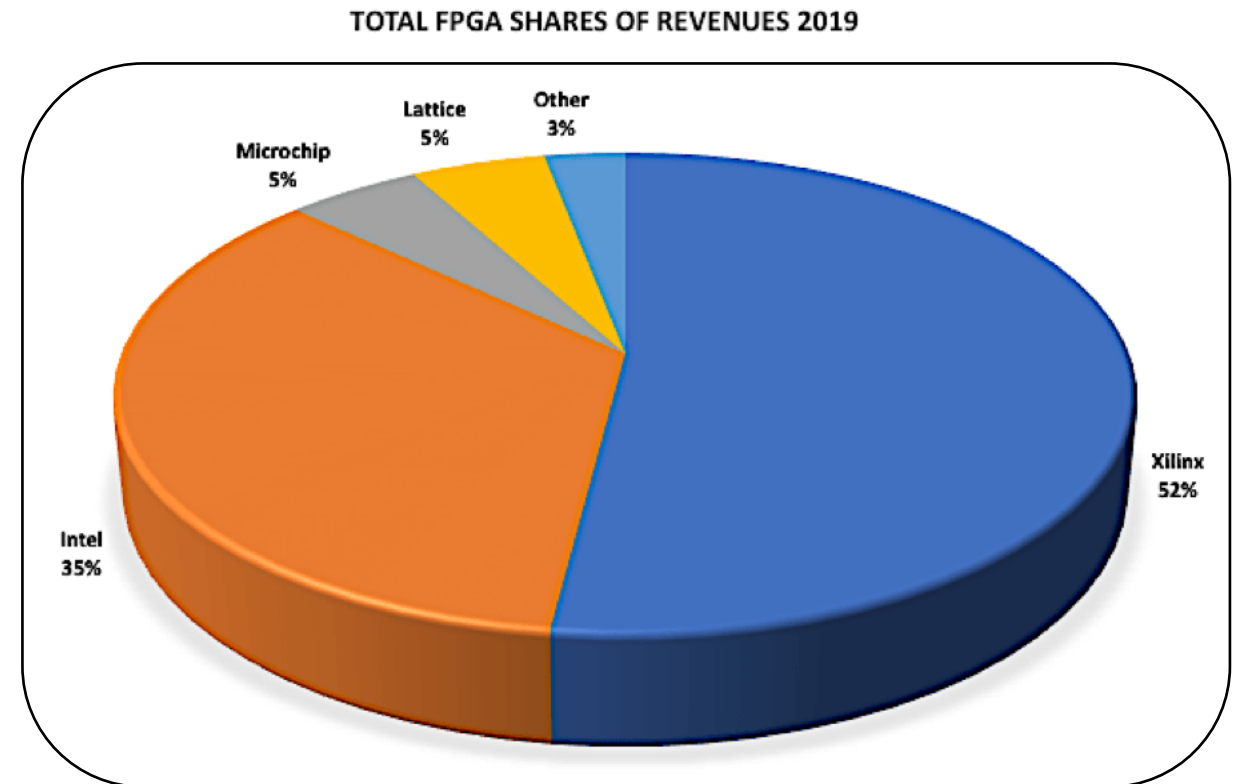
Considerando que são necessários cerca de 150 transistores para criar uma LUT4, tem-se para 50.000 LUTS (um FPGA médio pode ter tantas LUTs) → 7.500.000 transistores!!!

O MERCADO DE DISPOSITIVOS RECONFIGURÁVEIS

➤ Segmento de PLDs, 2009



➤ Segmento de FPGAs, 2019



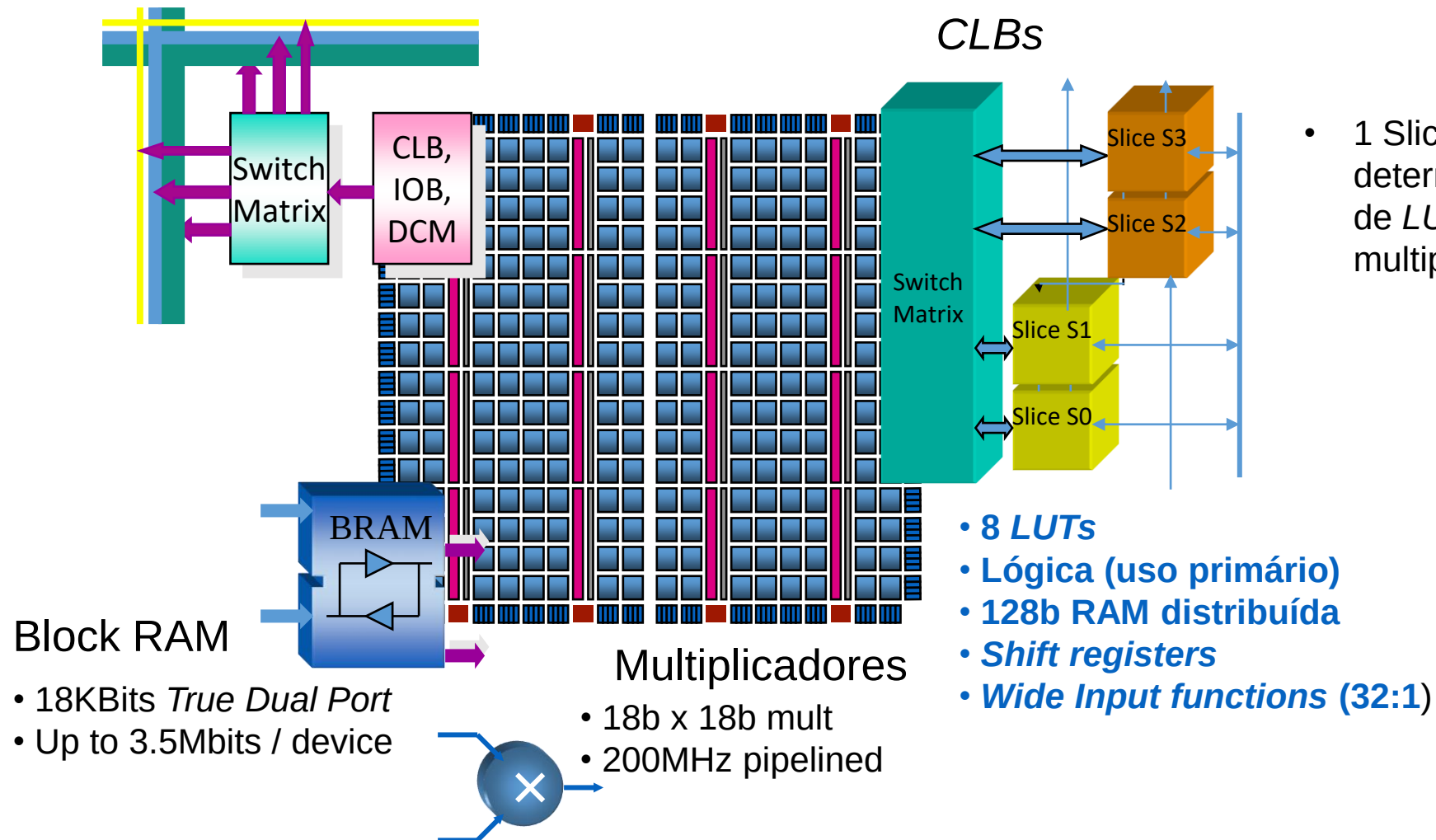
Source: The Information Network (www.theinformationnet.com)

- ✓ Apenas 2 fornecedores dominam o mercado, indicando que este está maduro

DISPOSITIVOS AMD E INTEL ATUAIS

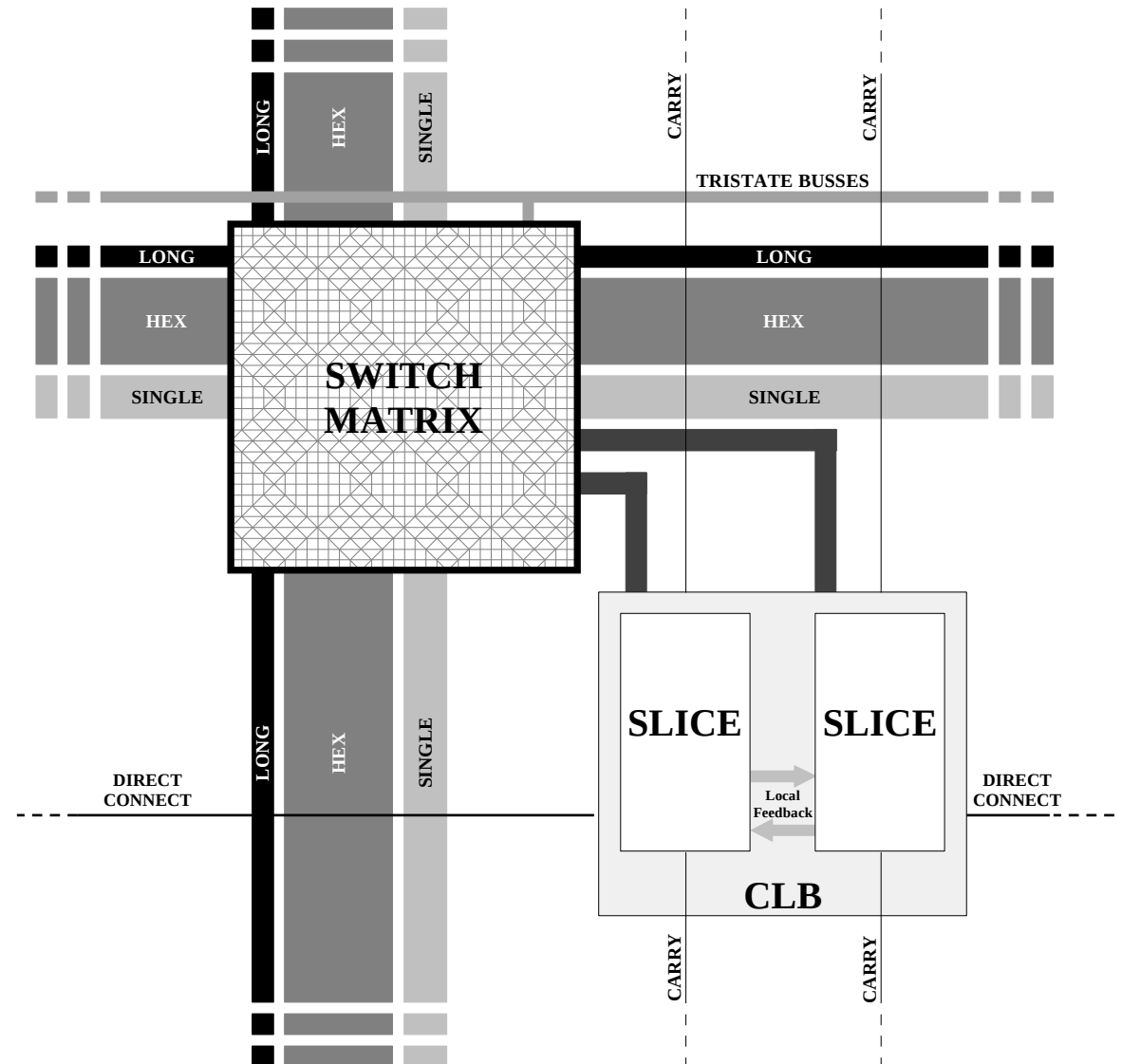
- **AMD (até 2022 era a empresa Xilinx, então adquirida pela AMD, por 60 GUS\$ em ações)**
 - ✓ Baixo/Médio Custo-Desempenho
 - Famílias Spartan 6, Spartan 7, Artix 7, Kintex 7...
 - **Spartan 3 e Spartan 3E (Temos placas, Nexys e Nexys2 com FPGA desta família)**
 - ✓ Alto desempenho
 - Virtex 7 (28nm), Virtex-Kintex UltraSCALE (20nm), Virtex-Kintex UltraSCALE+ (**16nm**), Versa (**7nm**)
- **Intel (até 2015 era a empresa Altera, então adquirida pela Intel, por 16,7 GUS\$)**
 - ✓ Baixo Custo e Mid-Range
 - Famílias Cyclone
 - **Max10 (Temos placa, MAX10, com FPGA desta família), Arria...**
 - ✓ Alto desempenho
 - Stratix, Agilex (**10nm**)...

ALGUNS DETALHES: FAMÍLIA VIRTEX II (2001-2002)



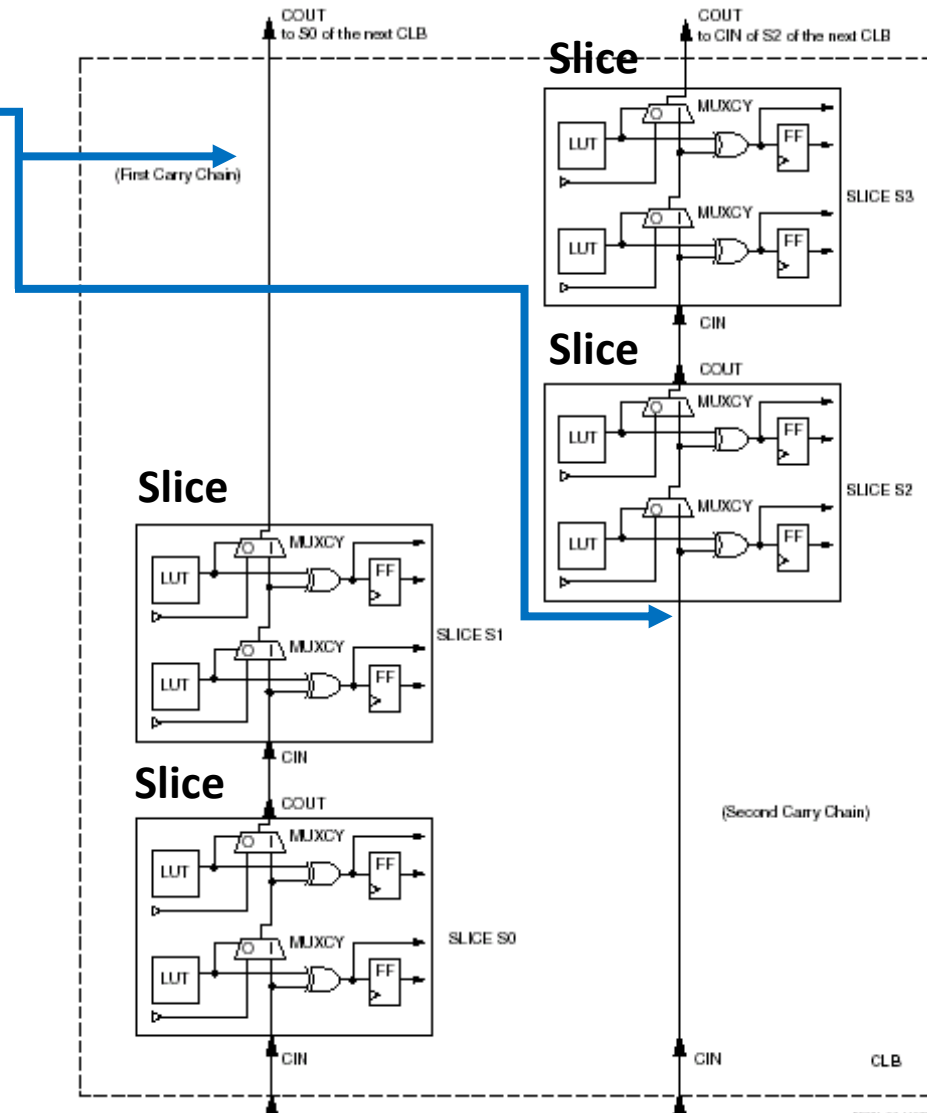
ARQUITETURA VIRTEX II – CLB E INTERCONEXÃO

- Conexões diretas entre *CLBs* vizinhas
 - ✓ Lógica de vai-um
- Matrix de conexão
 - ✓ *CLB* às linhas de roteamento
- Linhas de roteamento
 - ✓ Simples
 - ✓ Hexas
 - ✓ Longas
 - ✓ *Tri-state*

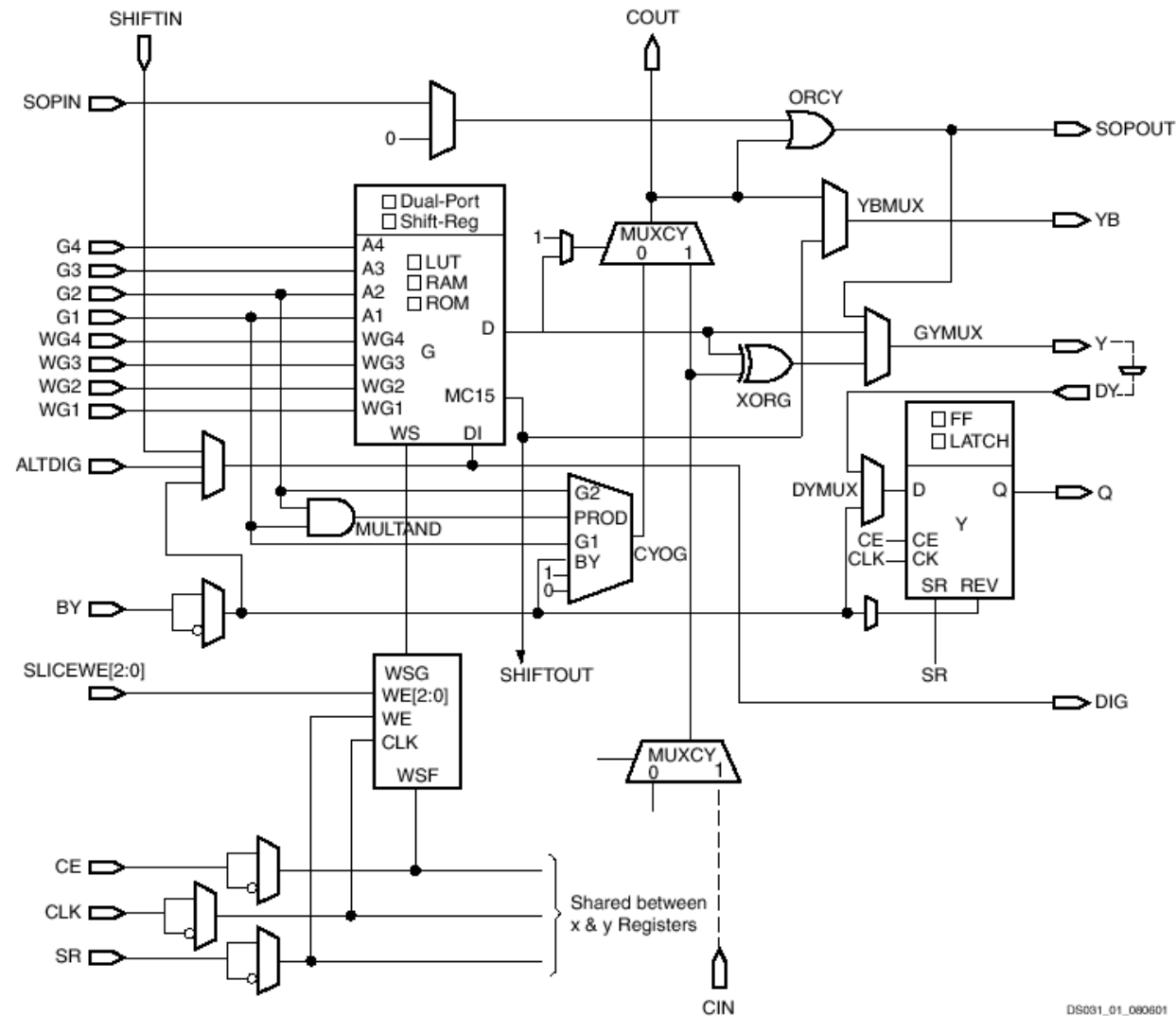


ARQUITETURA DO CLB DO DISPOSITIVO VIRTEX II

- *Fast Carry Logic Path*
- *Provides fast arithmetic (+, -, *, /)*
- Resumindo, um CLB na Virtex-II contém
 - ✓ 4 Slices
 - ✓ 8 LUTS / 8 Flip-Flops
 - ✓ 2 cadeias de propagação rápida de vai-um
 - ✓ 64 bits para compor uma memória (RAM/ROM)
 - ✓ 64 bits para fazer *shift registers*



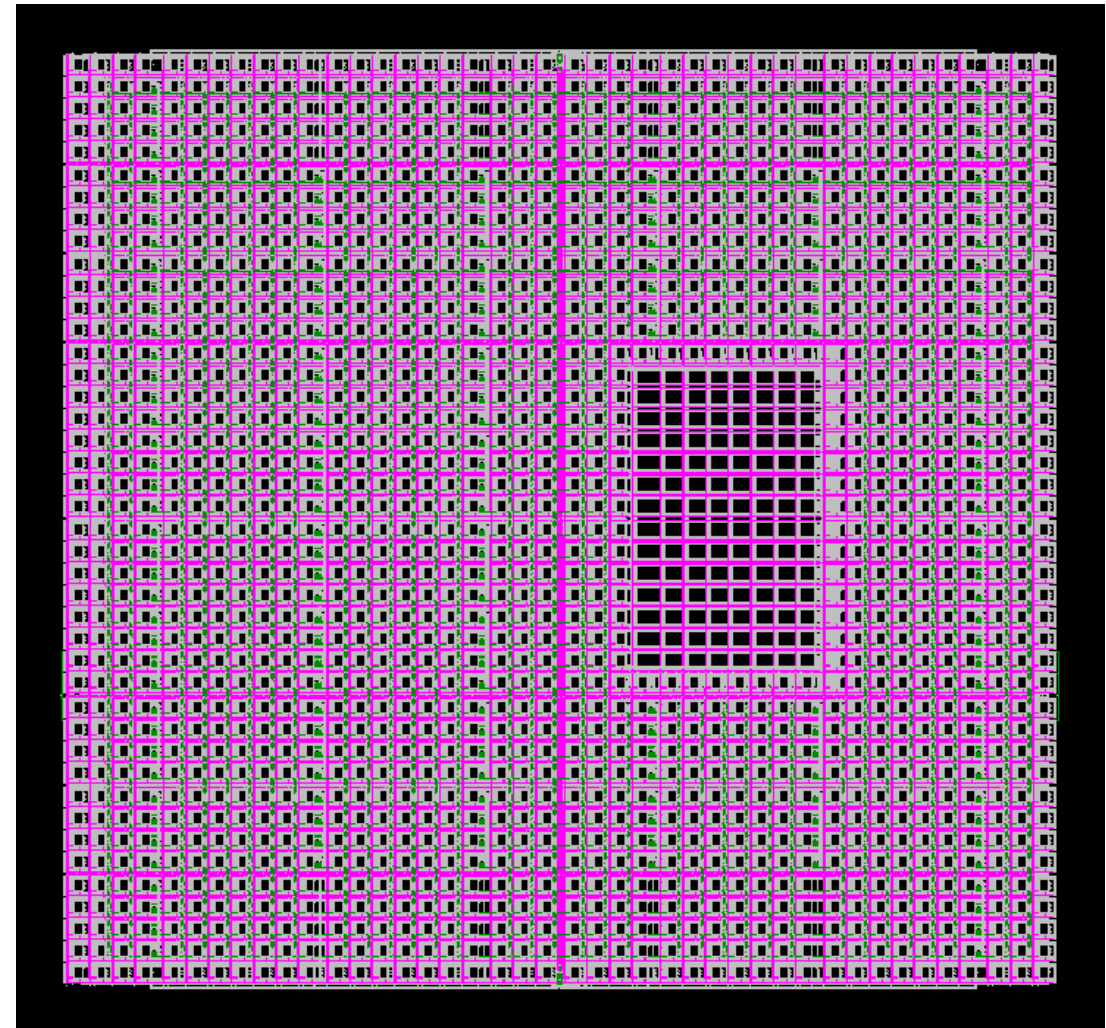
ARQUITETURA – METADE DE UMA SLICE



Um Exemplo: FAMÍLIA VIRTEX2P XC2VP7

➤ FPGA Editor View With All Wires

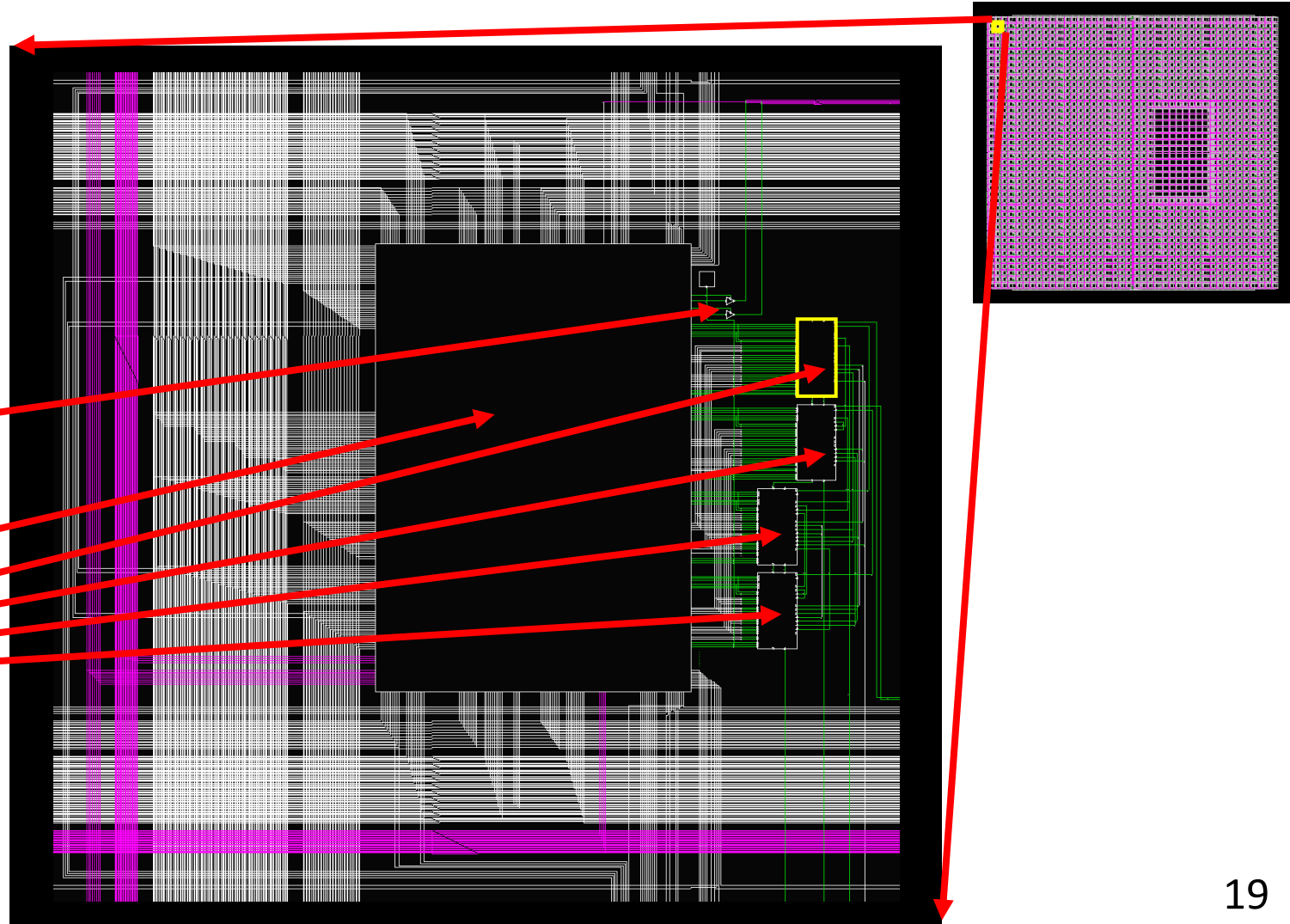
- ✓ 4,928 *slices*
- ✓ 44 *BRAMs*
- ✓ 1 PowerPC (microprocessador)
- ✓ 11,627 *logic sites*
- ✓ 2,653 *tiles*
- ✓ 1,423,681 *wires*
- ✓ 544,549 *wire segments (short)*



Um Exemplo: VIRTEX2P XC2VP7

➤ FPGA Editor View With All Wires

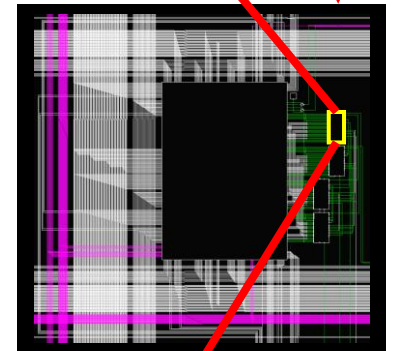
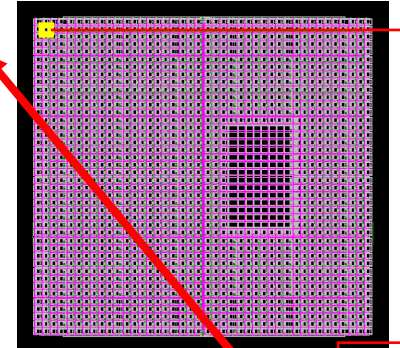
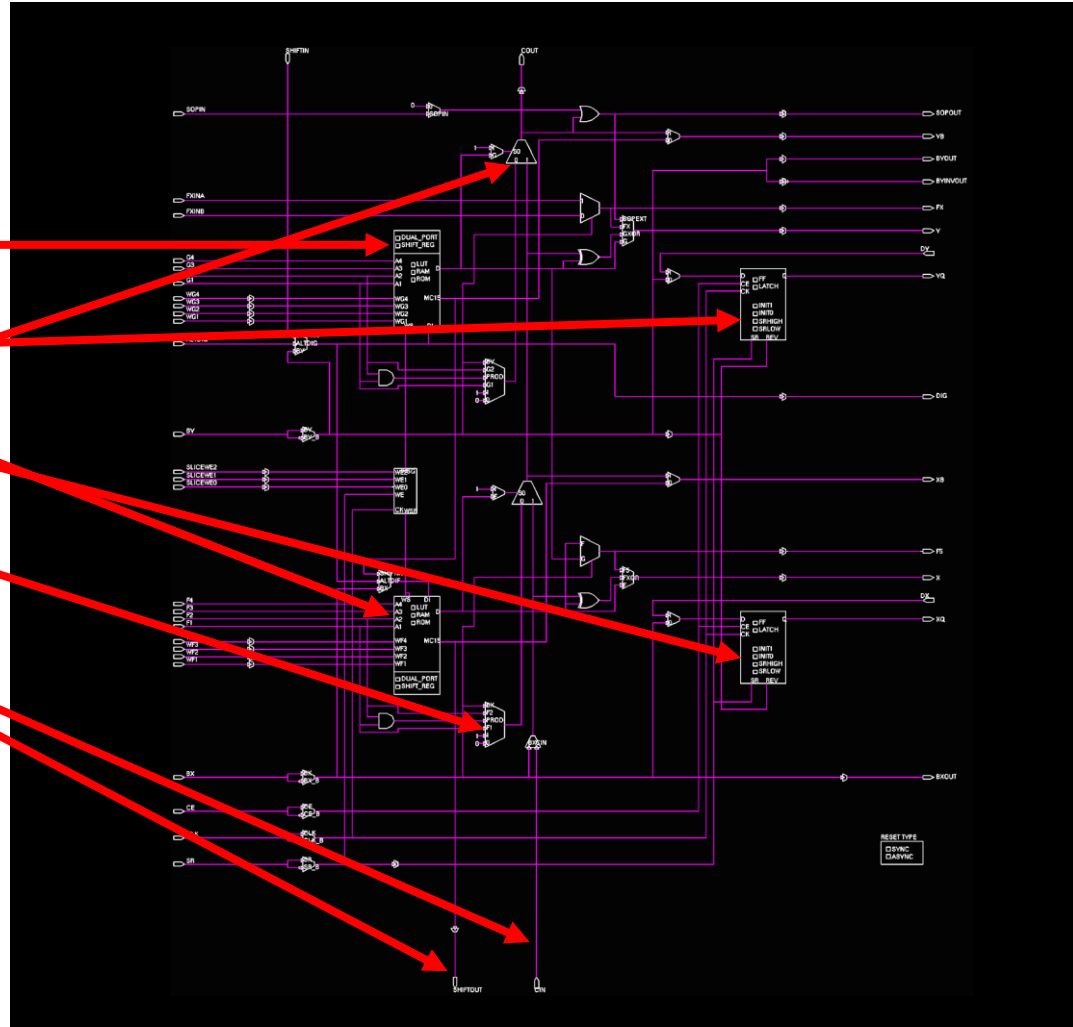
- ✓ Zoom de um *CLB* do canto superior esquerdo
- ✓ Muitos recursos de roteamento
- ✓ 2 *TBUFs*
- ✓ Grande caixa de conexões (*switch box*)
- ✓ 4 *slices*



Um Exemplo: VIRTEX2P XC2VP7

➤ FPGA Editor View With All Wires

- ✓ 1 Slice da Família *Virtex2Pro*
 - 2 LUTs
 - 2 flip-flops
 - Vários muxes
 - Lógica de vai-um dedicada



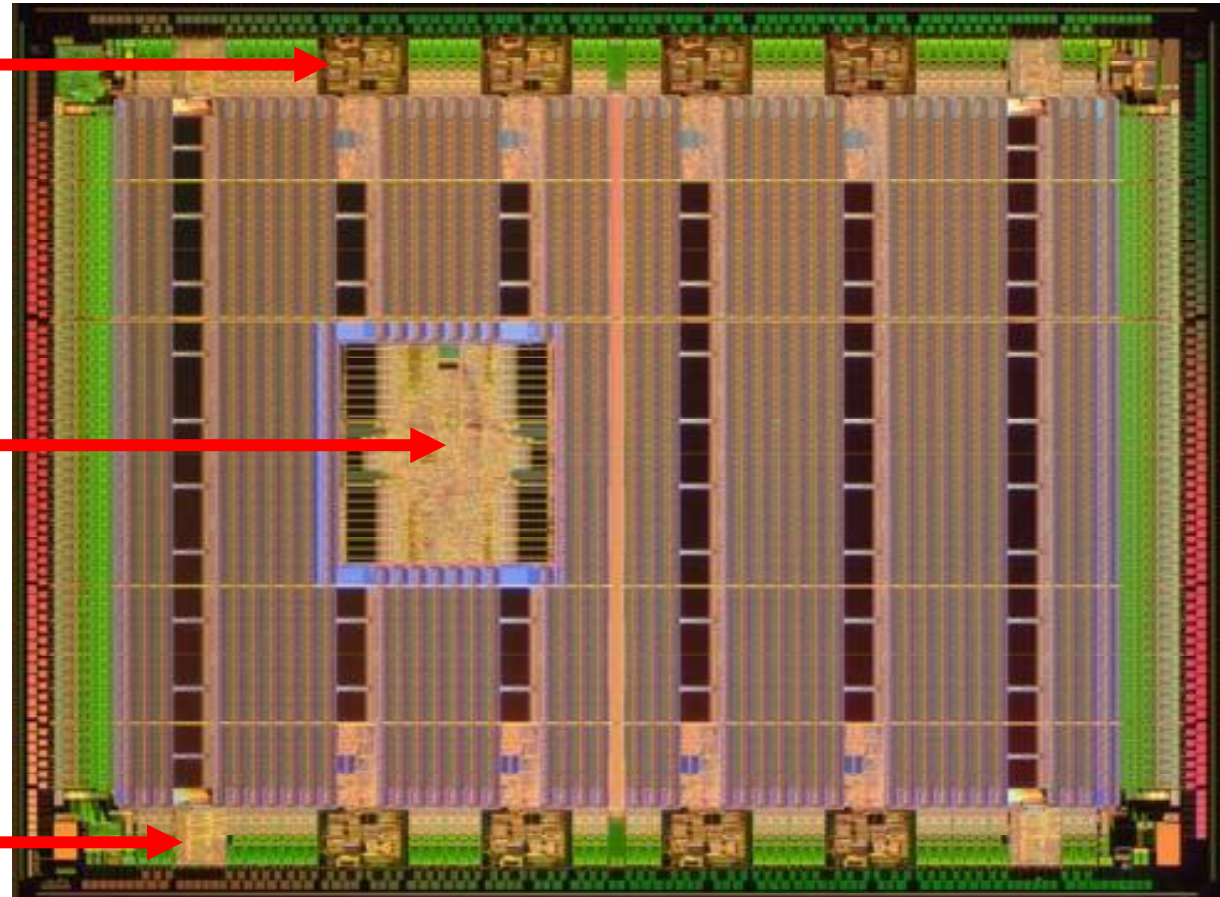
Um Exemplo de FPGA: o 2VP7 VIRTEX-II-PRO

➤ Foto do Layout do XC2VP7

✓ MGTs (Multi-Gigabit Transceivers)

✓ Power PC

✓ DCM (Digital Clock Manager)



DEMAIS COMPONENTES DE UM FPGA MODERNO

- Vários circuitos prontos existem dentro do FPGA para o projetista usar (*Hard IPs*), e.g.
 - Para gerenciamento do(s) sinal(ais) de relógio (*clock*)
 - ✓ Reduzem escorregamento de relógio
 - ✓ Permite multiplicar, dividir, mudar a fase da(s) frequências de entrada
 - ✓ Implementações digitais (DCM – [Xilinx](#), mais baratos) e analógica (PLL – [Altera](#), mais flexíveis)
 - Blocos de memória
 - ✓ Tipicamente blocos de 9Kbits, 18kbits ou 36Kbits na Xilinx (Altera tem mais variedade de tamanhos de blocos de memória)
 - Blocos DSP
 - ✓ Multiplicadores 18bitsX18bits configuráveis, muito usados para funções de imagem, áudio, telecomunicações, IA
- Outros circuitos existem já prontos para configurar no FPGA, de graça (os mais simples), ou por um preço (os mais complexos). Estes são denominados *Soft IPs*

DEMAIS COMPONENTES DE UM FPGA MODERNO

➤ Hard IPs

- ✓ Processadores embarcados do tipo hard IP
 - ✓ Xilinx disponibiliza hoje o processador ARM em algumas famílias (e.g. Zynq)
 - ✓ Podem executar sistemas operacionais embarcados como Linux
- ✓ Criptografia do *bitstream*
- ✓ Controle para reconfiguração interna (de dentro do FPGA - ICAP)

➤ Soft IPs

- ✓ Medium Access Control (MAC) para redes Ethernet e/ou outros tipos de rede
 - ✓ MACs são protocolos da camada 2 (modelo OSI/ISO – Redes)
 - ✓ Outros tipos de MAC existem para Bluetooth, WiFi, Zigbee etc.
- ✓ Processadores Soft – e. g. MicroBlaze ([Xilinx](#)), Nios ([Altera](#)) etc.
- ✓ Módulos de criptografia – AES, RSA etc.

INTRODUÇÃO À SIMULAÇÃO DE HARDWARE DIGITAL SÍNCRONO EM VHDL

INTRODUÇÃO A VHDL

- VHDL → Uma linguagem para *descrever* sistemas digitais
- Existem diversas outras: **SystemC**, **Verilog**, **SystemVerilog**, **Handel-C**, **ISP** etc...
- Praticamente VHDL e [System]Verilog são as mais usadas (sugestão, estudem [System]Verilog também!!)
- Origem
 - ✓ **VHDL**: **V**HSIC **H**ardware **D**escription **L**anguage
 - Criado no programa do DoD dos EUA: “*Very High Speed Integrated Circuits*” (VHSIC), de 1980.
 - Uma das linguagem padronizadas pelo “*Institute of Electrical and Electronics Engineers*” (IEEE) em 1987, revisado em 1993, 2000, 2002, 2004, 2009,...
 - ✓ Feita para *especificar* hardware. Atualmente serve também para **simulação** e **síntese**!
 - ✓ Linguagem utilizada mundialmente por empresas de CAD, com grande adoção na Europa e Japão. Já **Verilog** é muito usada nos EUA e junto com **SystemVerilog**, muito usada em chip design

BENEFÍCIOS

➤ Especificação do sistema digital

- ✓ Projetos independentes da tecnologia (implementação física é postergada)
- ✓ Ferramentas de CAD compatíveis entre si
- ✓ Permite explorar, em um nível mais alto de abstração (em relação a diagramas de esquemáticos) diferentes alternativas de implementação
- ✓ Permite, através de simulação, verificar o comportamento do sistema digital

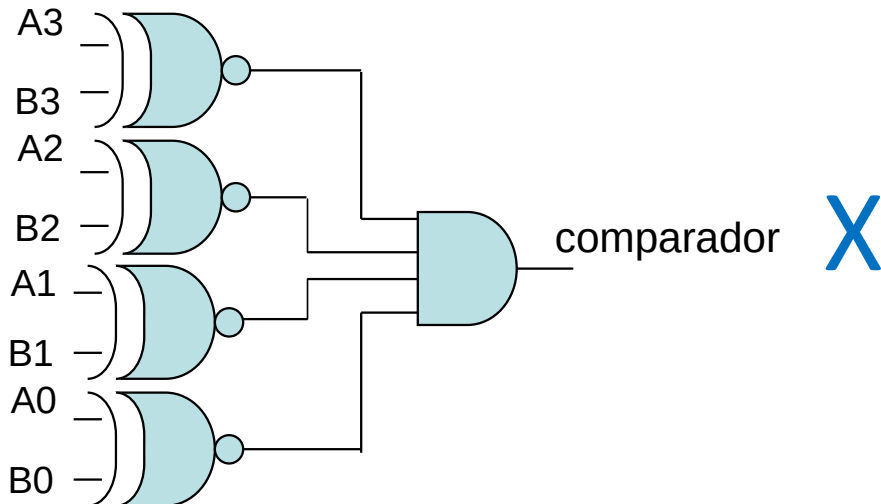
➤ Nível físico:

- ✓ Reduz tempo de projeto (favorece níveis abstratos de projeto)
- ✓ Reduz custo do projeto
- ✓ Elimina erros de baixo nível (se usado como base de ferramentas automatizadas)
- ✓ **Consequência:** reduz “*time-to-market*” (tempo de chegada de um produto ao mercado)

DESVANTAGENS

- Em relação a diagramas de esquemáticos
 - ✓ Hardware gerado pode ser menos otimizado
 - ✓ Controlabilidade / Observabilidade de projeto reduzidas

➤ Circuito Lógico



Fonte: Profa. Dra. Fernanda Gusmão de Lima Kastensmidt. Sistemas Digitais. UFRGS.

➤ VHDL

```
signal A : in  std_logic_vector(3 downto 0);  
signal B : in  std_logic_vector(3 downto 0);  
comparador : out  std_logic;  
...  
comparador <= '1' when A=B else '0';
```

VHDL É UMA LINGUAGEM DE PROGRAMAÇÃO?

➤ **NÃO.** Pois é uma linguagem de **descrição** de hardware!

➤ Código é executado em um *simulador*

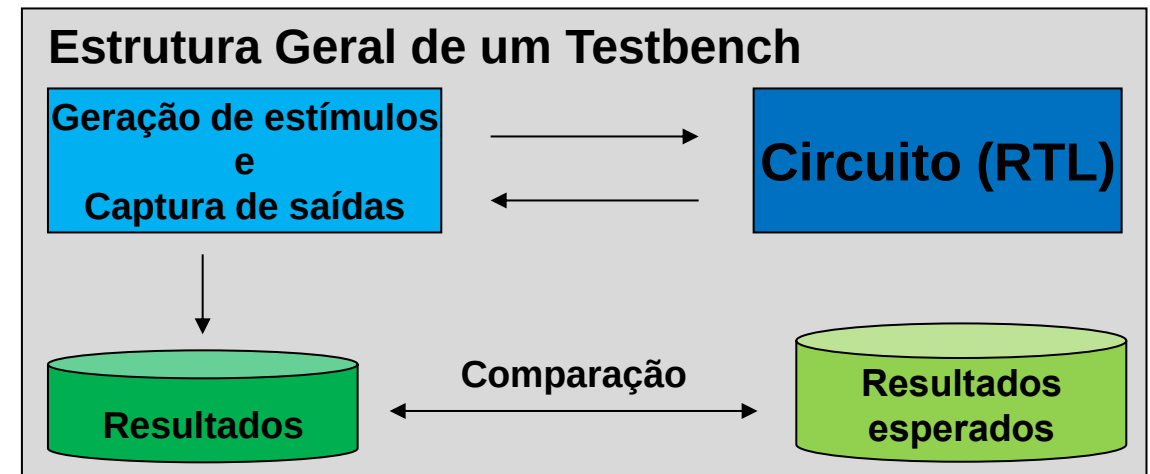
- ✓ Não se enxerga o “compilador” de VHDL, não há um “código executável” visível

➤ Projeto do usuário

- ✓ Especificado no nível de abstração denominado de transferência em registradores (em inglês, *Register – Transfer Level* ou **RTL**), mas não apenas neste nível!

➤ **Testbench** = descrição em VHDL (ou outra linguagem) do procedimento de teste de um circuito

- ✓ Especificação comportamental do ambiente externo ao projeto (estímulos externos)
- ✓ Interage com o projeto
- ✓ Não precisa ser descrito em VHDL (o projeto VHDL pode ser validado em ambiente C/C++!!)



TIPO PADRÃO PARA SÍNTESE

➤ standard_ulogic

- ✓ Tipo não “resolvido”
- ✓ Enumeração com 9 níveis lógicos

```
-- Fonte: stdlogic.vhd
-----
-- logic state system  (unresolved)
-----
TYPE std_ulogic IS ( 'U',  -- Uninitialized
                    'X',  -- Forcing  Unknown
                    '0',  -- Forcing  0
                    '1',  -- Forcing  1
                    'Z',  -- High Impedance
                    'W',  -- Weak      Unknown
                    'L',  -- Weak      0
                    'H',  -- Weak      1
                    '-'   -- Don't care
                );
```

TIPO PADRÃO PARA SÍNTESE

➤ standard_logic

- ✓ Tipo “resolvido” - permite por exemplo implementação de barramentos
- ✓ Tipo mais utilizado em descrições de hardware – Vejam abaixo como ele é definido formalmente, em VHDL

```
SUBTYPE std_logic IS resolved std_ulogic;
TYPE std_logic_vector IS ARRAY ( NATURAL RANGE <>) OF std_logic;
-----
-- resolution function
-----
CONSTANT resolution_table : stdlogic_table := (
--
-- | U   X   0   1   Z   W   L   H   -   |
--
-- |-----|
-- | ('U', 'U', 'U', 'U', 'U', 'U', 'U', 'U', 'U', 'U'), -- | U |
-- | ('U', 'X', 'X', 'X', 'X', 'X', 'X', 'X', 'X', 'X'), -- | X |
-- | ('U', 'X', '0', 'X', '0', '0', '0', '0', '0', 'X'), -- | 0 |
-- | ('U', 'X', 'X', '1', '1', '1', '1', '1', '1', 'X'), -- | 1 |
-- | ('U', 'X', '0', '1', 'Z', 'W', 'L', 'H', 'X'), -- | Z |
-- | ('U', 'X', '0', '1', 'W', 'W', 'W', 'W', 'X'), -- | W |
-- | ('U', 'X', '0', '1', 'L', 'W', 'L', 'W', 'X'), -- | L |
-- | ('U', 'X', '0', '1', 'H', 'W', 'W', 'H', 'X'), -- | H |
-- | ('U', 'X', 'X', 'X', 'X', 'X', 'X', 'X', 'X'), -- | - |
-- );
```

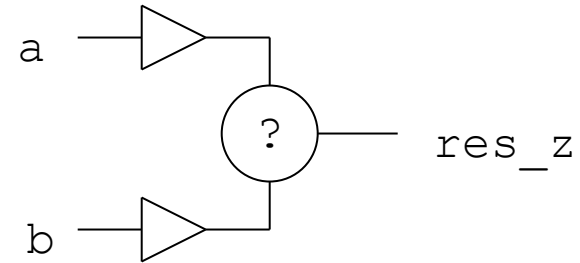
TIPO PADRÃO PARA SÍNTESE

```
signal a, b, z :<tipo pré-definido, não “resolvido”>;  
signal res_z : <tipo_resolvido>;
```

```
z <= a;  
z <= b;
```

X

- ✓ Erro de “*multiple drivers*”,
ou seja, curto-circuito

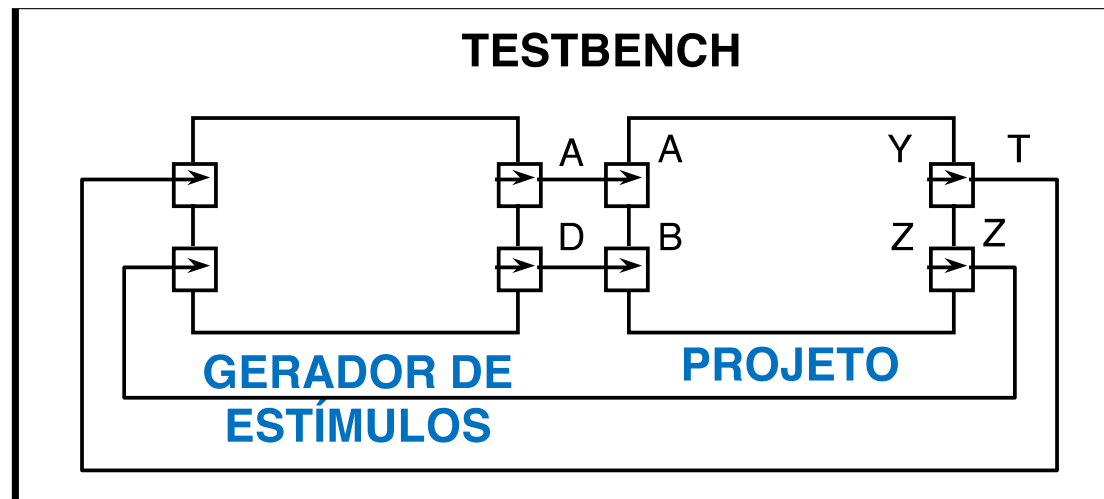


```
res_z <= a;  
res_z <= b;
```

- ✓ Resolução por tabela pré-definida
na linguagem ou em biblioteca da
linguagem (como um header file
em C ou Java)

VALIDAÇÃO POR SIMULAÇÃO

- **Testbenches** - Uma forma simples de testar o projeto
 - ✓ Na sua forma mais simples, um *testbench* consiste em um ou mais “processos geradores de estímulos” e uma instância do projeto que se quer testar
 - ✓ O *testbench* é construído como um módulo VHDL que não contém portas de entrada/saída. Ou seja, trata-se de um sistema fechado, ou autônomo!



COMO DESCREVER HARDWARE

➤ **Entity** – Interface Externa

- ✓ Especifica somente a interface entre o hardware e o ambiente
- ✓ Não contém definição do comportamento ou da estrutura internos do Hw

Código VHDL da entidade

```
entity halfadd is  
port (A, B: in std_logic;  
       Sum, Carry: out std_logic);  
end halfadd;
```

Esquemático da Interface

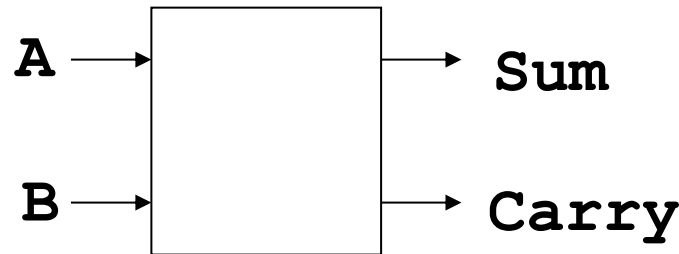


Tabela Verdade

A	B	Sum	Carry
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

COMO DESCREVER HARDWARE

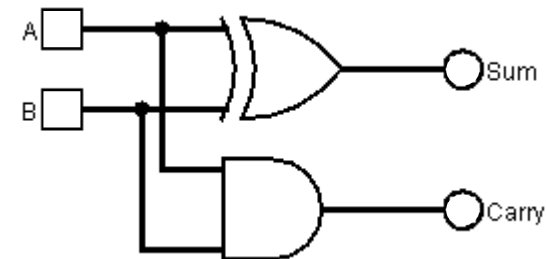
➤ **Architecture** – Comportamento do Hw

- ✓ Especifica o comportamento e/ou a estrutura internos de uma *entity*
- ✓ Deve ser associada a exatamente uma *entity* específica
- ✓ Uma *entity* pode ter associada a si várias *architecture* (representando diferentes formas de implementar um mesmo Hw)

Esquemático equivalente desta arquitetura

Código VHDL da arquitetura

```
architecture comp of halfadd is  
begin  
    Sum    <= A xor B;  
    Carry  <= A and B;  
end comp;
```



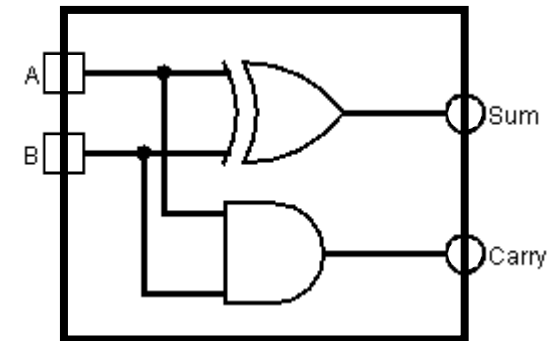
UMA DESCRIÇÃO COMPLETA DE HARDWARE

```
library IEEE;
use IEEE.std_logic_1164.all;

entity halfadd is
port (A, B: in std_logic;
      Sum, Carry: out std_logic);
end halfadd;

architecture comp of halfadd is
begin
    Sum    <= A xor B;
    Carry <= A and B;
end comp;
```

Esta biblioteca e o pacote definem o tipo `std_logic`, o qual não é parte da linguagem VHDL!



EXEMPLO DE TESTBENCH PARA UM HARDWARE

```
library IEEE;
use IEEE.std_logic_1164.all;

entity halfadd_tb is
end halfadd_tb;

architecture TB_ARCHITECTURE of halfadd_tb is
    signal aa, bb, soma, vaium : std_logic;
begin

    UUT : entity work.halfadd
        port map ( A => aa, B => bb,
                  Sum => soma, Carry => vaium );

    g1: aa <= '0', '1' after 10 ns, '0' after 20 ns, '1' after 30 ns;
    g2: bb <= '0', '1' after 20 ns;

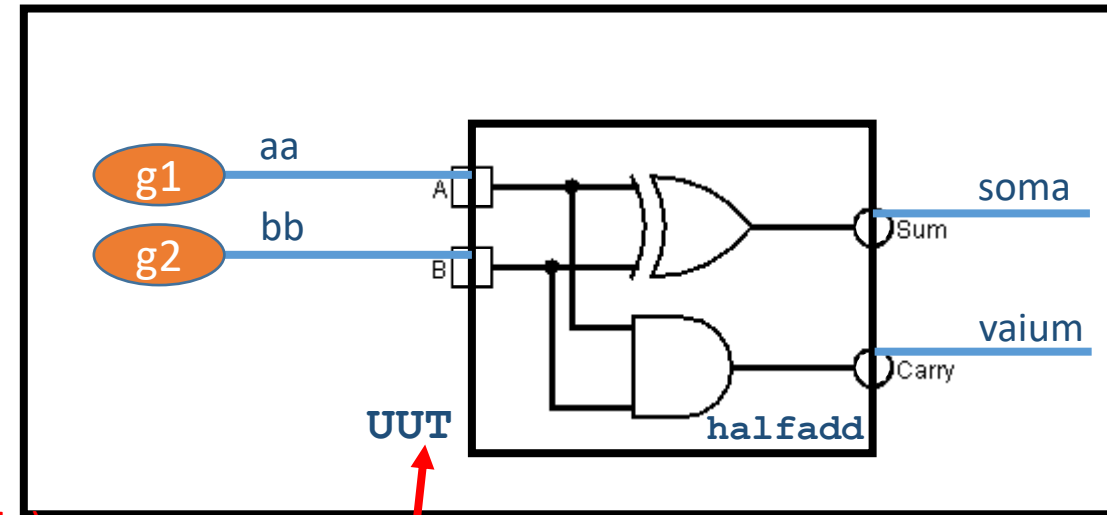
end TB_ARCHITECTURE;
```

Testbench não tem pinos
externos (ports in ou out)

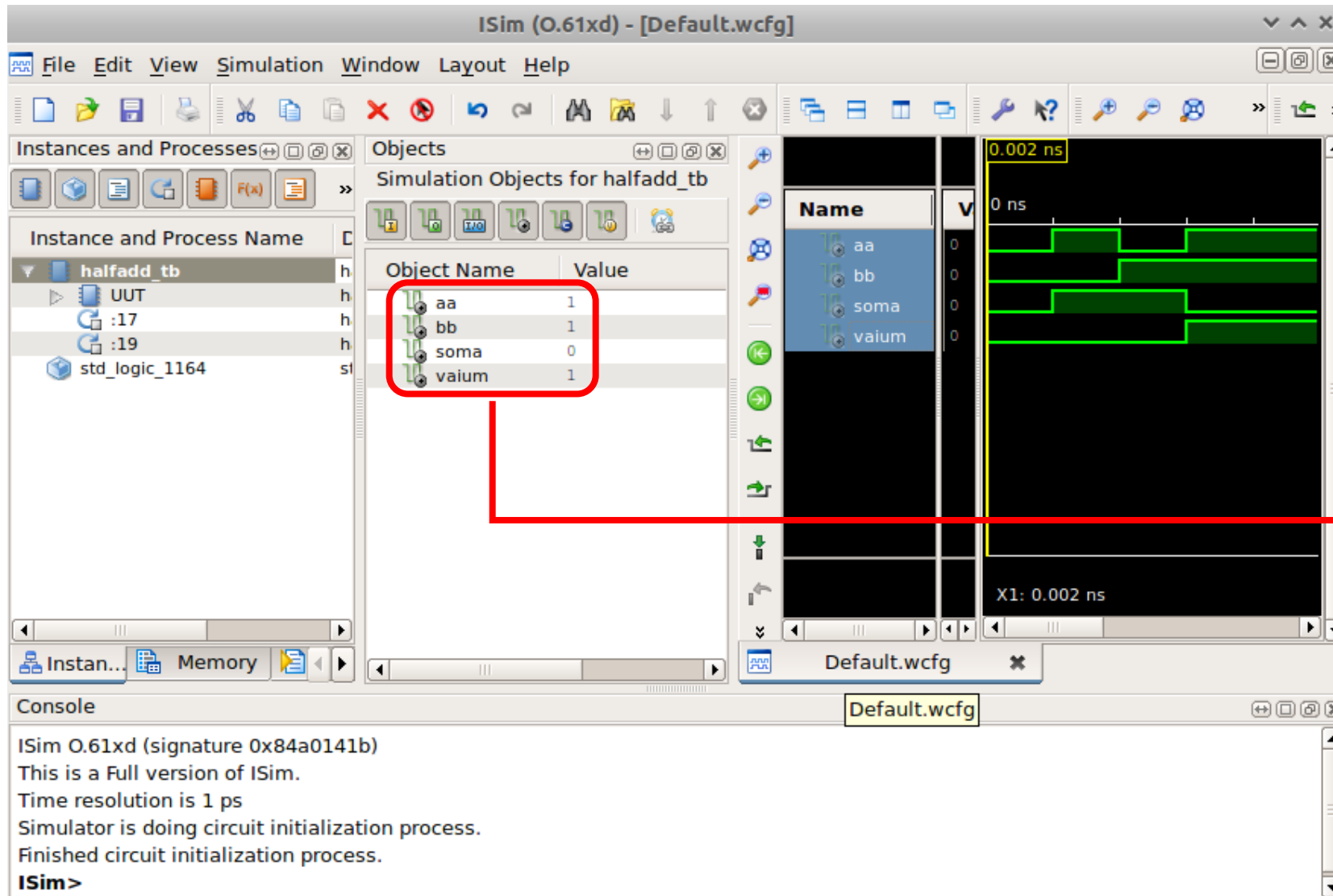
Nome do projeto (entidade)

Instanciação do projeto,
conectando pinos do
projeto aos fios do testador

Geração dos estímulos,
dizendo como fios se
comportam



EXEMPLO DE SIMULAÇÃO DE UM HARDWARE



TUTORIAL 1: CAPTURA DE PROJETO E SIMULAÇÃO DE HARDWARE (XILINX ISIM)

CRIANDO UM AMBIENTE DE EXECUÇÃO

- Hoje, usar localmente a infraestrutura da PUCRS não é uma opção viável na quantidade de vezes que a disciplina requer
- Há várias opções para ter acesso ao ambiente Vivado, por exemplo
 1. Instalar um software de acesso a servidores da PUCRS (Linux)
 - ✓ O software **xpra** é uma excelente opção, gratuita e poderosa
 2. Instalar uma máquina virtual Linux contendo o VIVADO
 - ✓ Funciona bem mas precisa de uma boa máquina e muito disco
 3. Instalar o VIVADO no Windows
 - ✓ Forma simples, mas requer bastante disco/SSD ~45Gbytes
- Reflitam sobre a disponibilidade individual de cada um de vocês, escolham uma opção, o professor pode ajudar aqui!! Instalem uma das soluções em máquina que tenham à disposição de vocês
- Minha recomendação: Instalar Vivado localmente, veja como [aqui!](#)

USANDO O SIMULADOR DO VIVADO (V2020.2)

➤ Criando os arquivos do projeto

1. Criar um diretório de trabalho. Exemplo [half_adder](#)
2. Criar os dois arquivos fonte, com extensão VHDL

✓ somador1.vhd

```
library IEEE;
use IEEE.std_logic_1164.all;

entity halfadd is
port (A, B: in std_logic;
      Sum, Carry: out std_logic);
end halfadd;

architecture comp of halfadd is
begin
    Sum    <= A xor B;
    Carry <= A and B;
end comp;
```

✓ somador1_tb.vhd

```
library IEEE;
use IEEE.std_logic_1164.all;

entity halfadd_tb is
end halfadd_tb;

architecture TB_ARCHITECTURE of halfadd_tb is
    signal aa, bb, soma, vaium : std_logic;
begin

    UUT : entity work.halfadd
        port map ( A => aa, B => bb,
                  Sum => soma, Carry => vaium );

    aa <= '0', '1' after 10 ns, '0' after 20 ns, '1' after 30 ns;

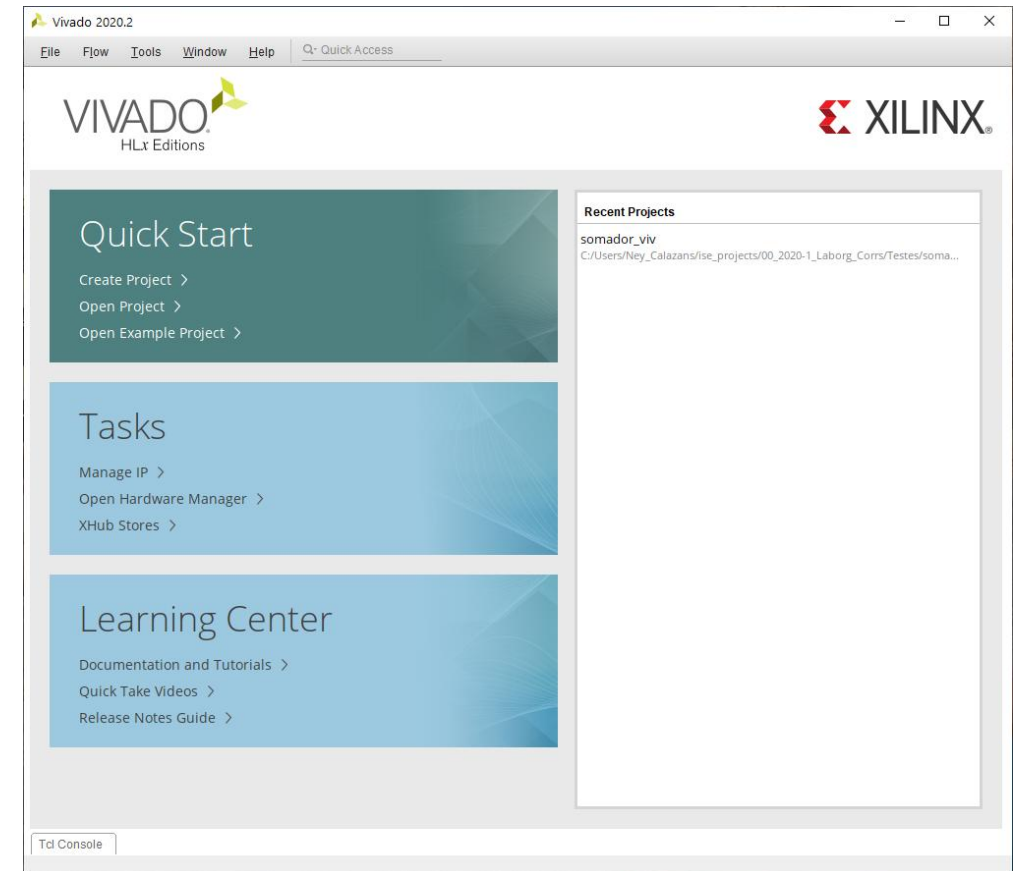
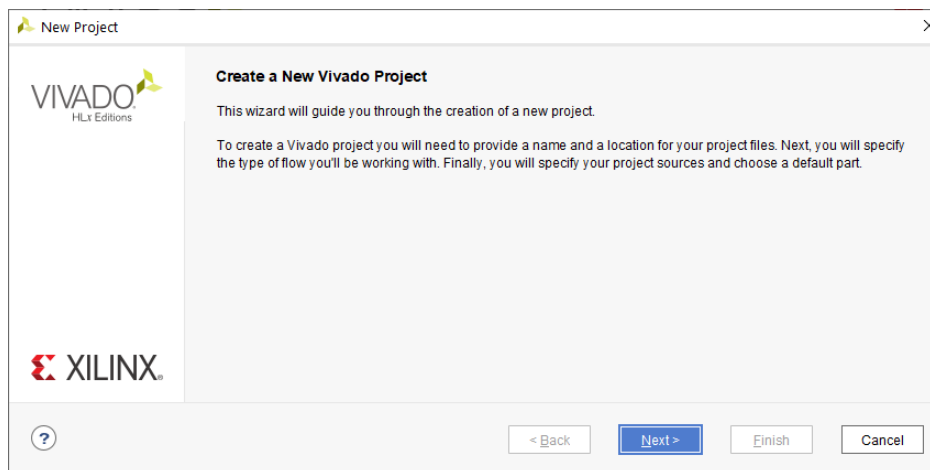
    bb <= '0', '1' after 20 ns;

end TB_ARCHITECTURE;
```

USANDO O SIMULADOR DO VIVADO (V2020.2)

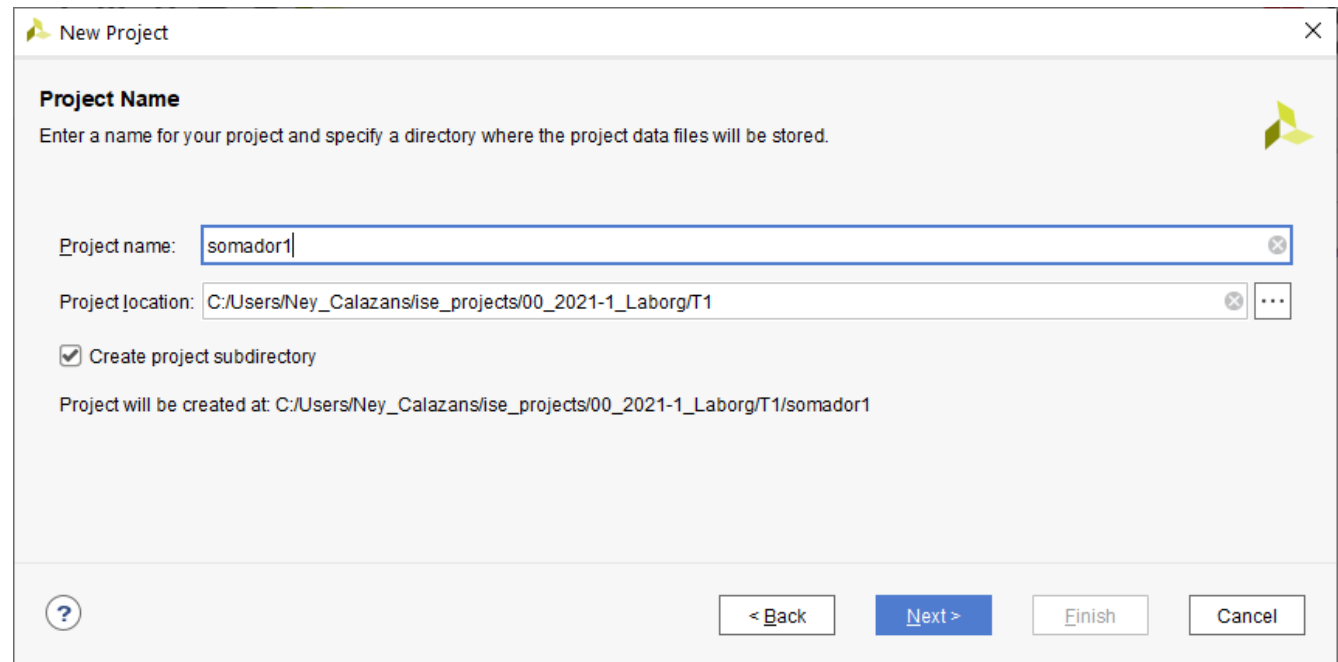
➤ Criando um projeto no ambiente VIVADO para o FPGA da Nexys A7

3. Caso algum projeto seja aberto ao lançar o VIVADO, feche-o com a opção de menu **File→Close Project**
4. Crie um novo projeto de nome **somador1** apertando o botão **Create Project**. (Lembre-se: escolha um diretório no qual tens direito de escrita). Leia o conteúdo da tela inicial e depois clique no botão **Next**.



USANDO O SIMULADOR DO VIVADO (V2020.2)

- **Criando um projeto no ambiente VIVADO para o FPGA da Nexys A7**
5. Preencha os campos **Project name** e **Project location**. Observação: Não usem caracteres especiais (acentos, brancos, etc.), seja no nome do projeto, seja no nome de qualquer parte do caminho. Garanta que a caixa ao lado de **Create project subdirectory** esteja marcada. Clique no botão **Next**.



New Project

Project Name
Enter a name for your project and specify a directory where the project data files will be stored.

Project name: somador1

Project location: C:/Users/Ney_Calazans/ise_projects/00_2021-1_Laborg/T1

☒ Create project subdirectory

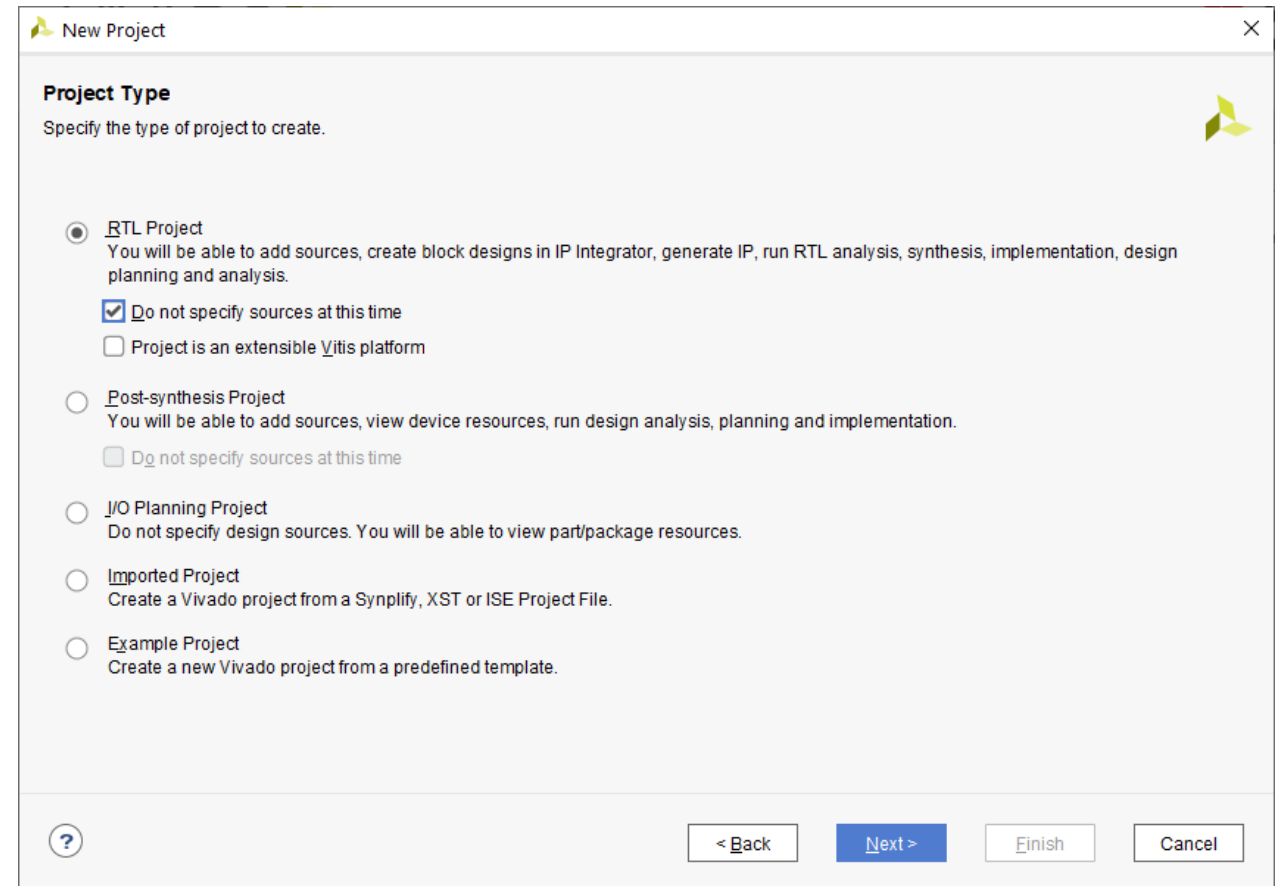
Project will be created at: C:/Users/Ney_Calazans/ise_projects/00_2021-1_Laborg/T1/somador1

? < Back Next > Finish Cancel

USANDO O SIMULADOR DO VIVADO (V2020.2)

➤ Criando um projeto no ambiente VIVADO para o FPGA da Nexys A7

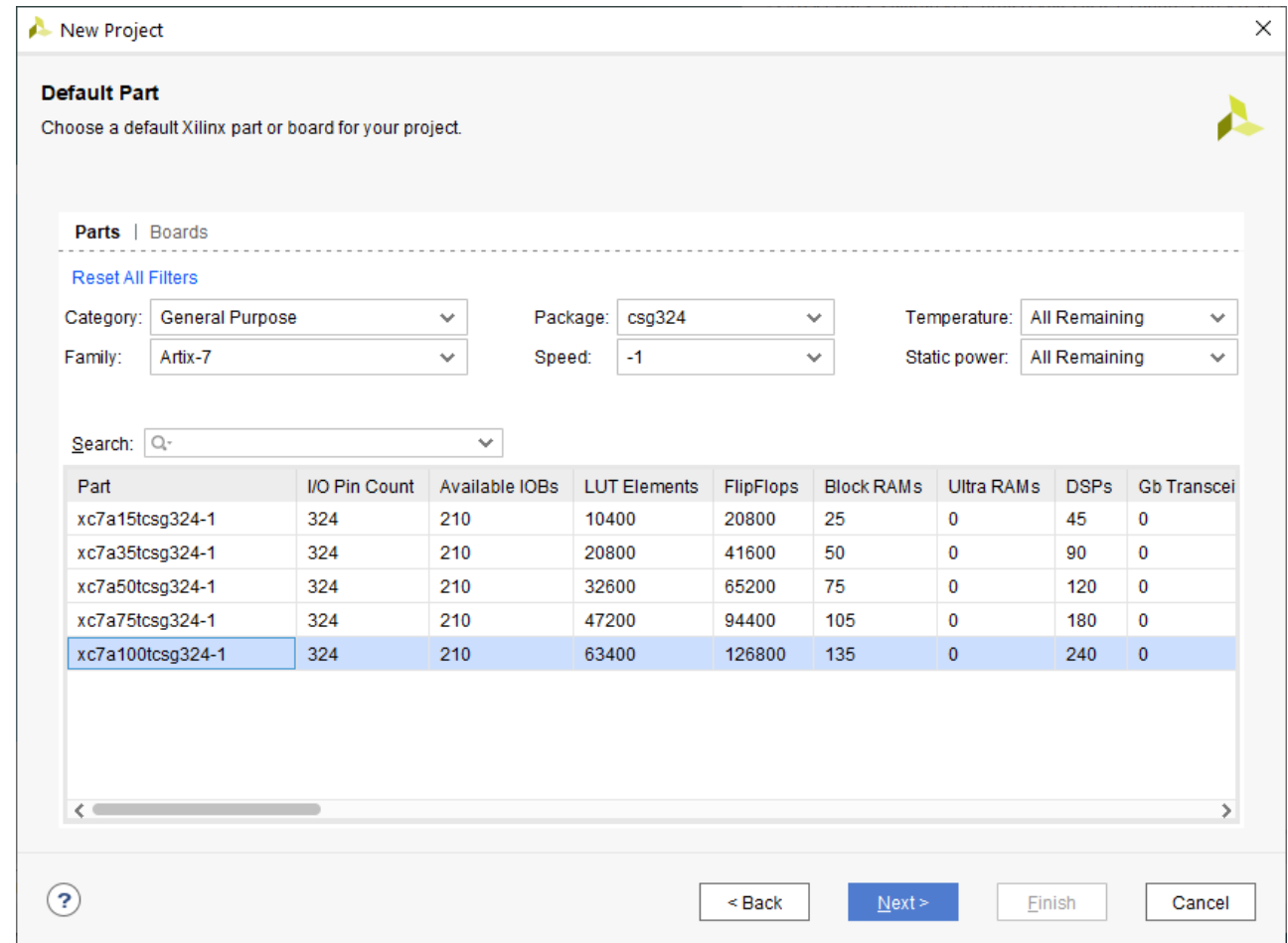
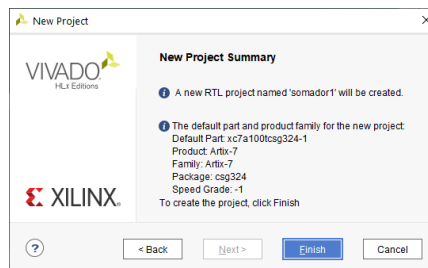
6. Na janela seguinte, escolha a opção **RTL Project**. RTL significa *Register Transfer Level*, ou Nível de Transferência entre Registradores o nível de abstração em que costumamos descrever hardware ao usar VHDL para descrever um circuito que queremos construir. Marque a caixa ao lado da opção **Do not specify sources at this time**, pois vamos acrescentar manualmente os arquivos fonte do projeto. Em seguida, clique no botão **Next**.



USANDO O SIMULADOR DO VIVADO (V2020.2)

➤ Criando um projeto no ambiente VIVADO para o FPGA da Nexys A7

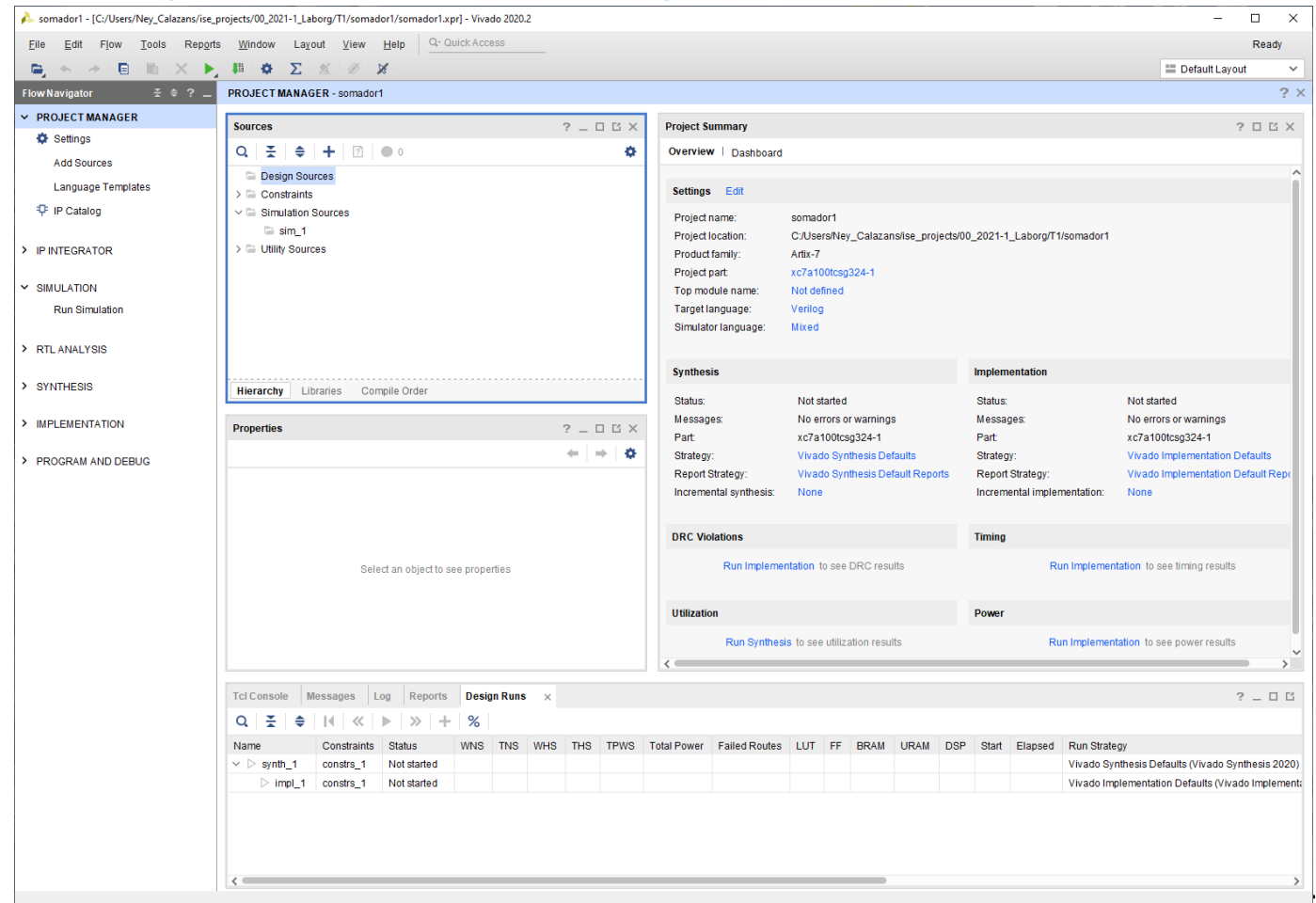
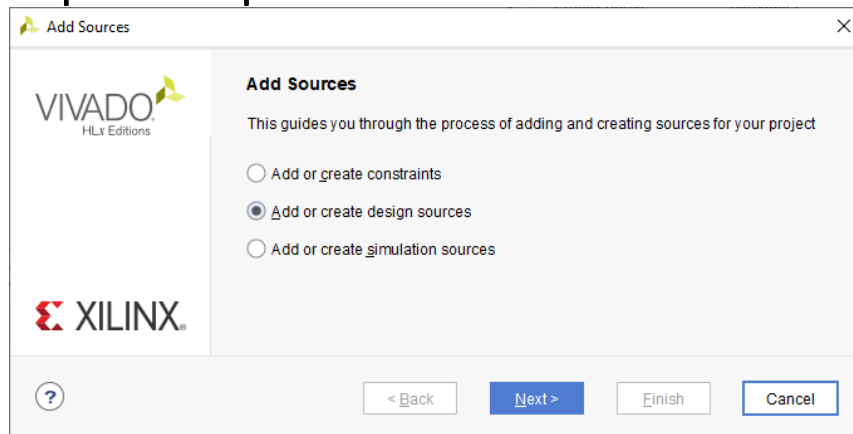
7. Na janela seguinte, faça as escolhas marcadas na Figura ao lado para achar o dispositivo da placa Nexys A7. Estas são **Category: General Purpose**; **Family: Artix-7**; **Package: csg324**; **Speed: -1**. Em seguida, clique no botão **Next**.
8. Confira os conteúdos da janela seguinte (**New Project Summary**) e depois clique no botão **Finish**.



USANDO O SIMULADOR DO VIVADO (V2020.2)

➤ Criando um projeto no ambiente VIVADO para o FPGA da Nexys A7

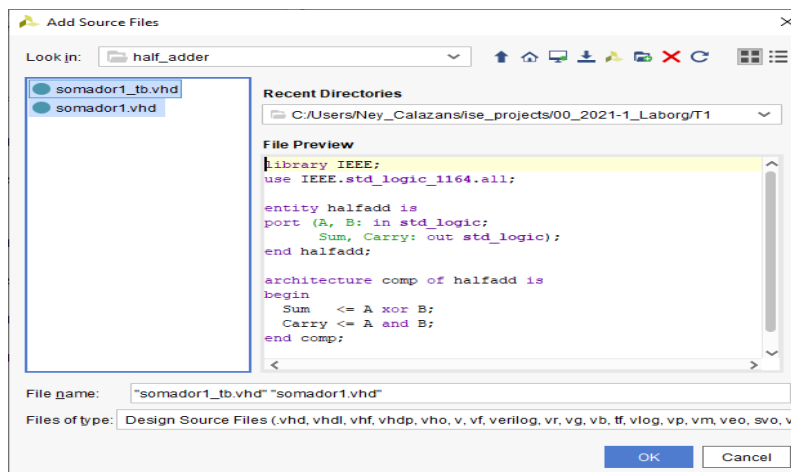
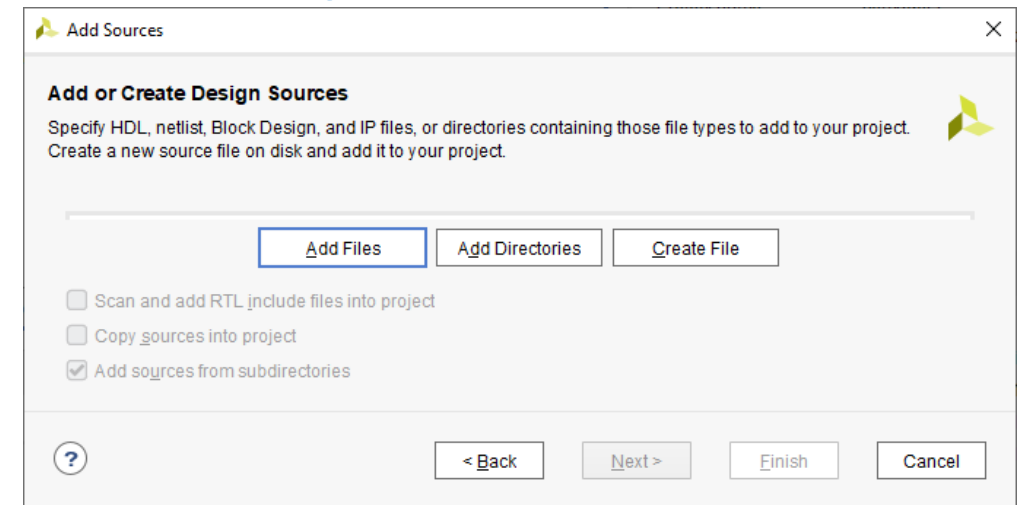
9. A janela ao lado deve ser similar ao que se vê. Na janela denominada **Sources** clique no símbolo **+** para acrescentar os arquivos fonte ao projeto.
10. Escolha a opção **Add or create design sources** na janela que aparece e depois clique em **Next**.



USANDO O SIMULADOR DO VIVADO (V2020.2)

➤ Criando um projeto no ambiente VIVADO para o FPGA da Nexys A7

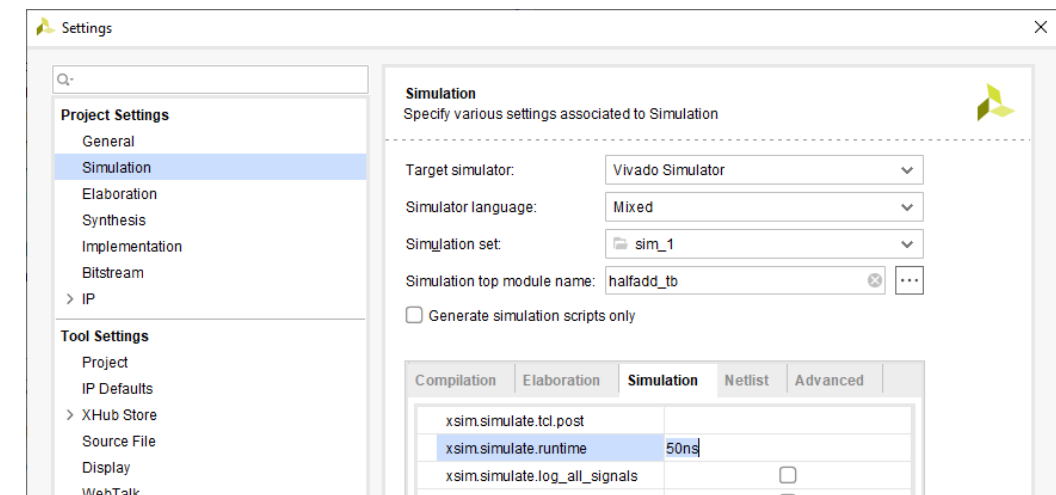
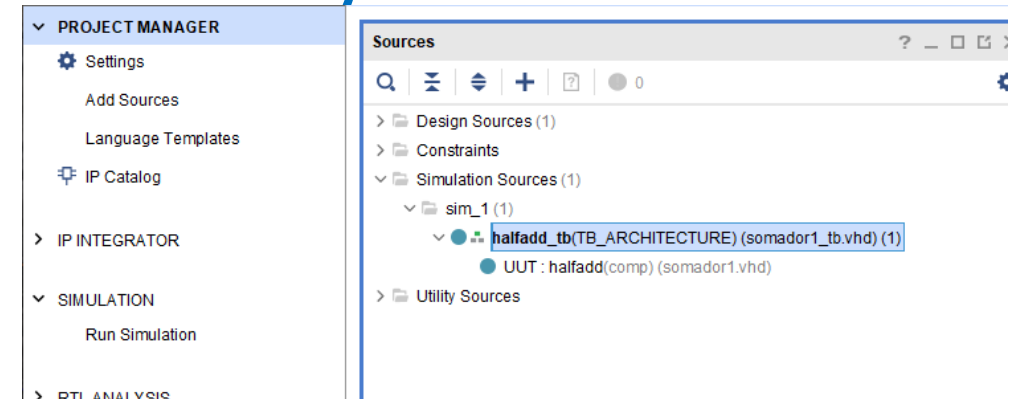
11. Na janela **Add Sources** clique em Add Files.
12. Na janela que surge, navegue até o diretório onde você criou os arquivos somador1.vhd e somador1_tb.vhd, selecione ambos e em seguida clique em **OK**.
13. Depois, ao voltar à janela **Add Sources** clique em **Finish**, que deve agora estar habilitado.



USANDO O SIMULADOR DO VIVADO (V2020.2)

➤ Criando um projeto no ambiente VIVADO para o FPGA da Nexys A7

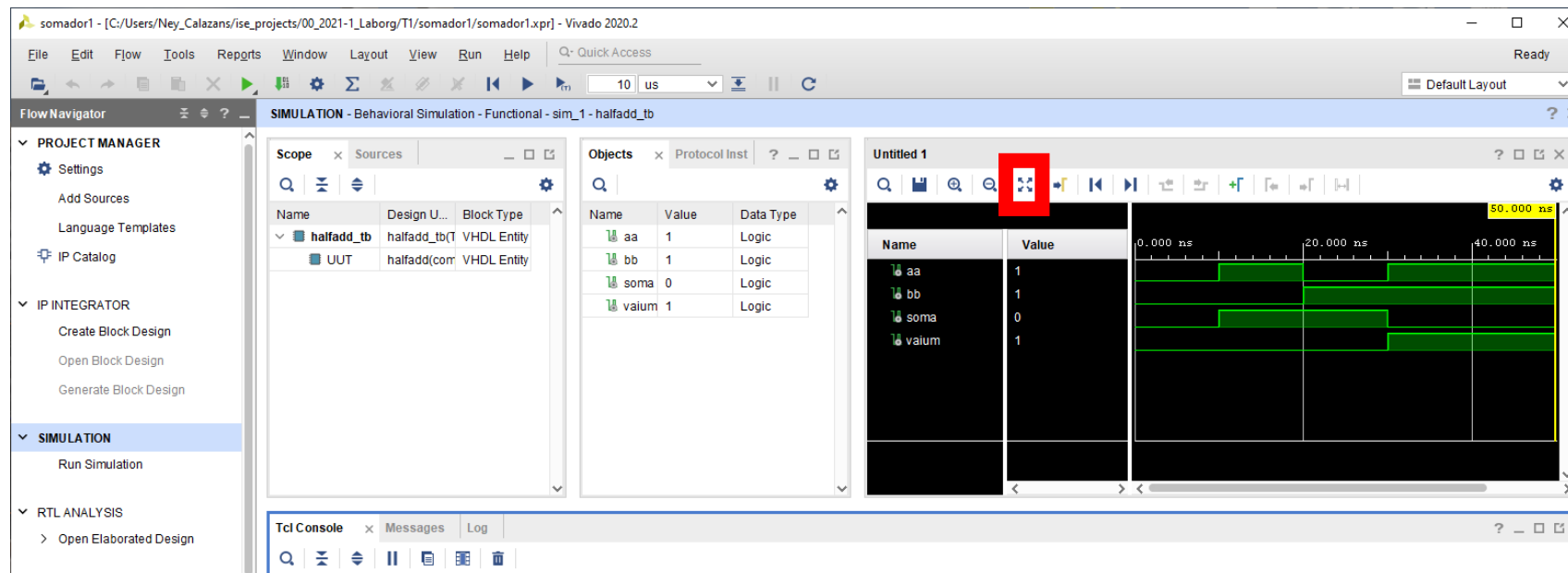
14. Na janela **Sources**, ache agora a entidade `halfadd_tb` (Sob o diretório **Simulation Sources (1)** e neste sob o sub-diretório `sim_1(1)`) e selecione ela.
15. Agora vamos configurar a simulação. Clique com o botão direito do mouse sobre o item **Run Simulation** sob a opção **SIMULATION** da janela **PROJECT MANAGER**. Em seguida escolha a opção **Simulation Settings** no menu *pop-up* que surge. Na janela que surge, selecione **Simulation** em **Project Settings**, e selecione a aba **Simulation** na janela da direita (**Simulation**). Na linha `xsim.simulate.runtime` troque 1000ns (ou o que lá houver) por 50ns. Finalmente, clique em **Apply** e **OK**.



USANDO O SIMULADOR DO VIVADO (V2020.2)

➤ Criando um projeto no ambiente VIVADO para o FPGA da Nexys A7

16. Agora, clique na janela **PROJECT MANAGER** no item **Run Simulation** sob a opção **SIMULATION** (desta vez com o **botão esquerdo** do mouse!). No diálogo de pop-up que surge, escolha a opção **Run Behavioral Simulation** (deve ser a única habilitada).
17. Para ver as formas de onda completas, clique no símbolo com 4 setas diagonais saindo do centro de um quadrado. Veja o resultado esperado abaixo.



USANDO O SIMULADOR DO VIVADO (V2020.2)

Note: Esta forma de onda mostra todas as linhas da tabela verdade do somador (4 combinações de entradas)

Observação: Lembrem-se de simular o testbench e não o projeto, selecionando o arquivo no topo da hierarquia. Para ver toda uma onda simulada na janela clique no botão “Zoom Fit”

The screenshot displays the Vivado 2020.2 simulation environment. The top menu bar includes File, Edit, Flow, Tools, Reports, Window, Layout, View, Run, and Help. The main workspace is divided into several panels:

- Flow Navigator:** Contains sections for PROJECT MANAGER (Settings, Add Sources, Language Templates, IP Catalog), IP INTEGRATOR (Create Block Design, Open Block Design, Generate Block Design), SIMULATION (Run Simulation), and RTL ANALYSIS (Open Elaborated Design).
- Scope:** Shows the simulation scope for 'halfadd_tb' and 'UUT'.
- Objects:** Displays the objects in the simulation, including 'aa', 'bb', 'soma', and 'vaum'.
- Waveform Viewer:** Shows the simulation results for the signals 'aa', 'bb', 'soma', and 'vaum'. The signals are plotted over time, with a 'Zoom Fit' button highlighted in a red box.

The waveform viewer shows the signals 'aa', 'bb', 'soma', and 'vaum' over time. The signals are plotted as digital waveforms, with the time axis ranging from 0.000 ns to 50.000 ns. The signals 'aa' and 'bb' are high (1) for most of the simulation, while 'soma' and 'vaum' are low (0) for most of the simulation.

TRABALHO T1

TRABALHO T1

- O **Trabalho T1** consiste em executar a simulação de 3 projetos no simulador do ambiente VIVADO da Xilinx
 - ✓ **Projeto 1**
 - Adaptação ao ambiente de simulação através do desenvolvimento de um meio-somador de 1 bit ([half_adder](#)). Este projeto já foi desenvolvido nos slides anteriores (Seção “Usando o Simulador do Ambiente VIVADO da Xilinx”) → apenas repita o processo descrito aqui anteriormente, para fixar o processo
 - ✓ **Projeto 2**
 - Desenvolvimento de um somador de 4 bits sem vai-um ([full_adder](#)) → simples
 - ✓ **Projeto 3**
 - Desenvolvimento de três implementações (arquiteturas) de um flip-flop mestre-escravo assíncrono ([ffd](#)) → exigente

PROJETO 2

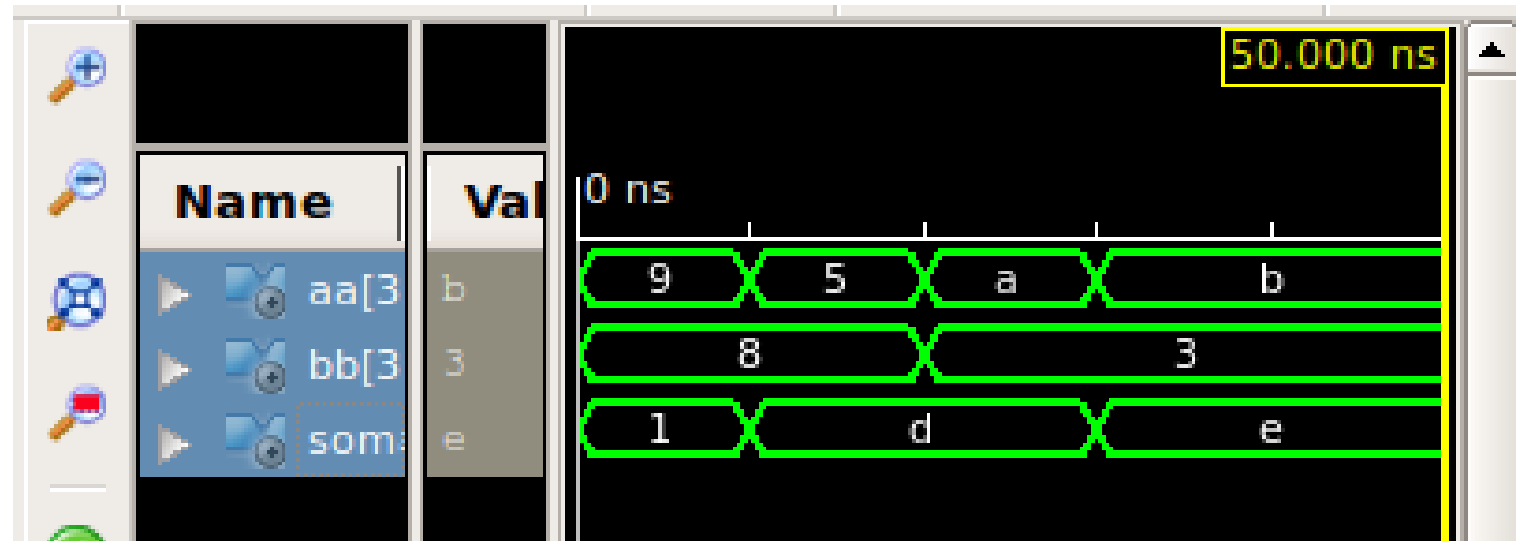
- Escreva um novo código (faça um novo projeto VHDL no VIVADO. Use a opção *Copy Project* do VIVADO) e utilize sinais não de 1 bit mas de 4 bits (`std_logic_vector(3 downto 0)`). O efeito é transformar o meio somador de 1 bit em um somador de 4 bits.
 - ✓ Note: `std_logic` → 1 fio e `std_logic_vector` → 1 ou mais fios (barramento)
- A nova entidade VHDL deve parecer com o seguinte

```
entity soma is
port (A, B: in std_logic_vector(3 downto 0);
      Soma: out std_logic_vector(3 downto 0));
end soma;
```
- O corpo da arquitetura VHDL deverá ser alterado e conter apenas `Soma <= A + B;`
- Deve-se incluir a linha: `use IEEE.std_logic_unsigned.all` (Porquê?)
- Escreva o *testbench* correspondente, estude como!

PROJETO 2

- Os 4 bits podem ser representados como um único dígito em hexadecimal. No *testbench* pode-se escrever
 - ✓ `aa <= x"9", x"5" after 10 ns, x"A" after 20 ns, x"B" after 30 ns;`
 - ✓ `bb <= x"8", x"3" after 20 ns;`

- Ao simular este circuito, deve aparecer uma forma de onda similar à apresentada na Figura à direita. Você pode testar somas de outros valores no seu *testbench*



- O comportamento é o esperado? **Explique, analisando cada combinação!! Duvide!!**

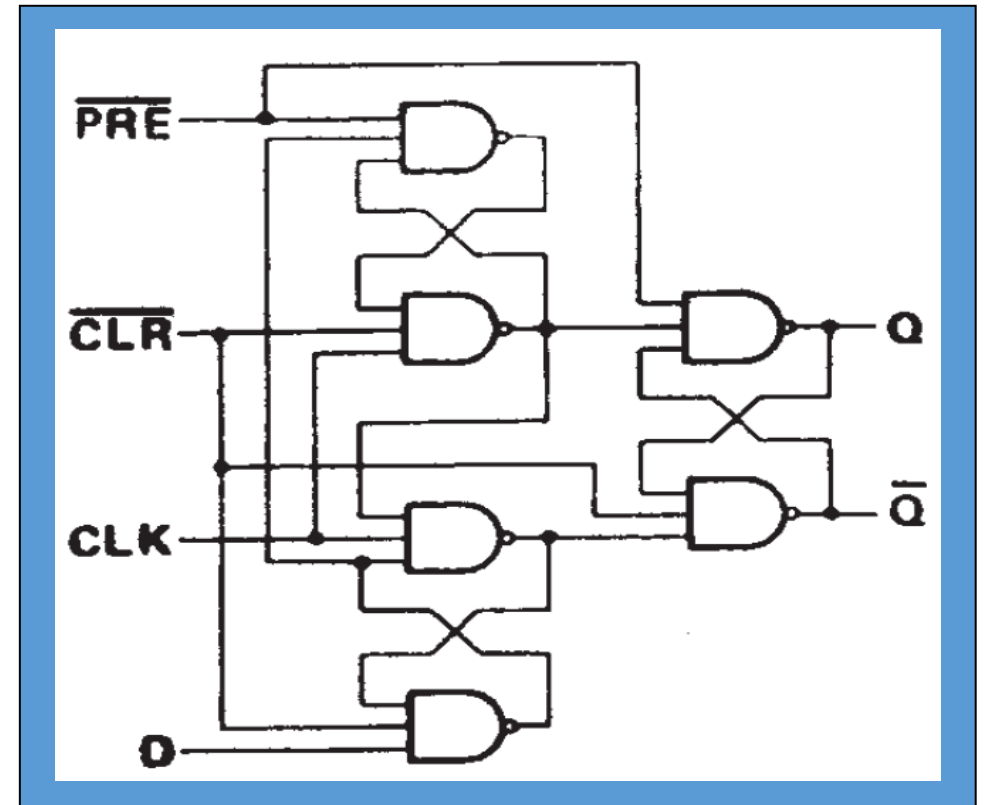
PROJETO 2

- Para gerar todas as combinações possíveis de entradas deste circuito seria necessário produzir 256 padrões (combinando os 16 valores distintos de A com os 16 valores distintos de B). Um trecho de código VHDL capaz de gerar estes estímulos é dado abaixo. Estude-o e entenda-o, simule-o se quiser

```
signal aa : std_logic_vector(3 downto 0) := "0000"; -- valor inicial
signal bb : std_logic_vector(3 downto 0) := "0000"; -- valor inicial
...
process (aa)
begin
    if (aa/=x"F") then
        aa <= aa+x"1" after 10ns; -- a cada 10ns
    else aa <= x"0" after 10ns; -- incrementa A
    end if;
end process;
process (bb)
begin
    if (bb/=x"F") then
        bb <= bb+x"1" after 160ns; -- a cada 16
    else bb <= x"0" after 160ns; -- valores de A,
    end if; -- incrementa B
end process;
```

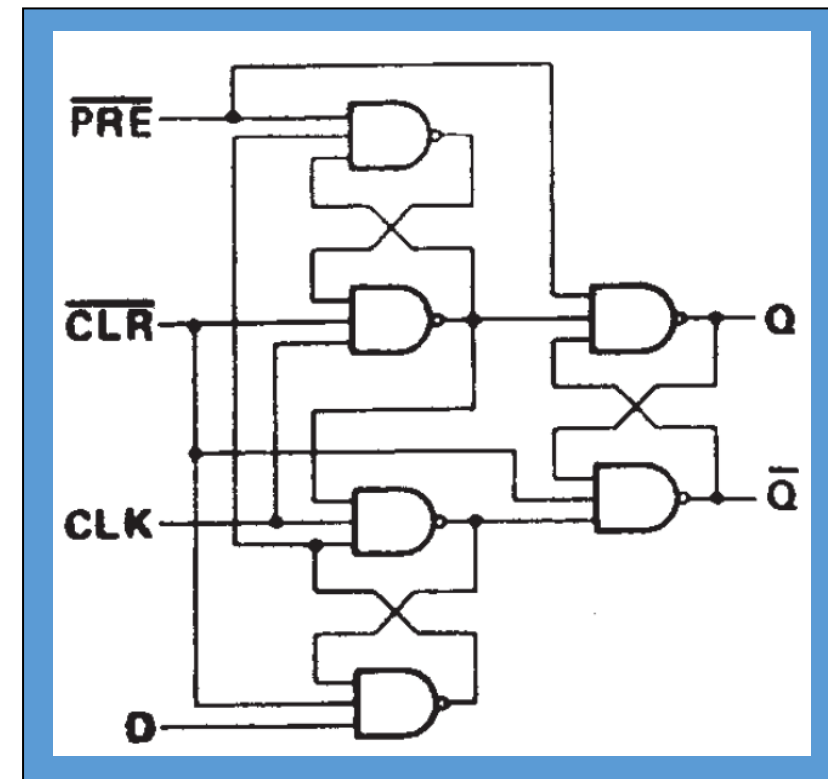
PROJETO 3

1. Implemente o circuito de seis portas lógicas ao lado em VHDL – aqui **é proibido usar o comando process de VHDL!** Este último comando será tema de aula posterior
2. Crie um projeto no ambiente VIVADO que vai conter a entidade (**entity**) correspondente a este circuito e três arquiteturas (**architecture**) descritas mais adiante
3. Crie um *testbench* para cada versão (**architecture**) do circuito. Este deve instanciar cada circuito, gerar estímulos para todas suas 4 entradas e aplicar os estímulos a instância da arquitetura em questão
4. Simule cada *testbench* e descreva textualmente como o circuito funciona
5. **Importante:** A descrição VHDL deve refletir exatamente a estrutura do diagrama de portas ao lado, não apenas seu comportamento! Por isto a proibição de usar **process**, que serve para descrever comportamento, não estrutura.



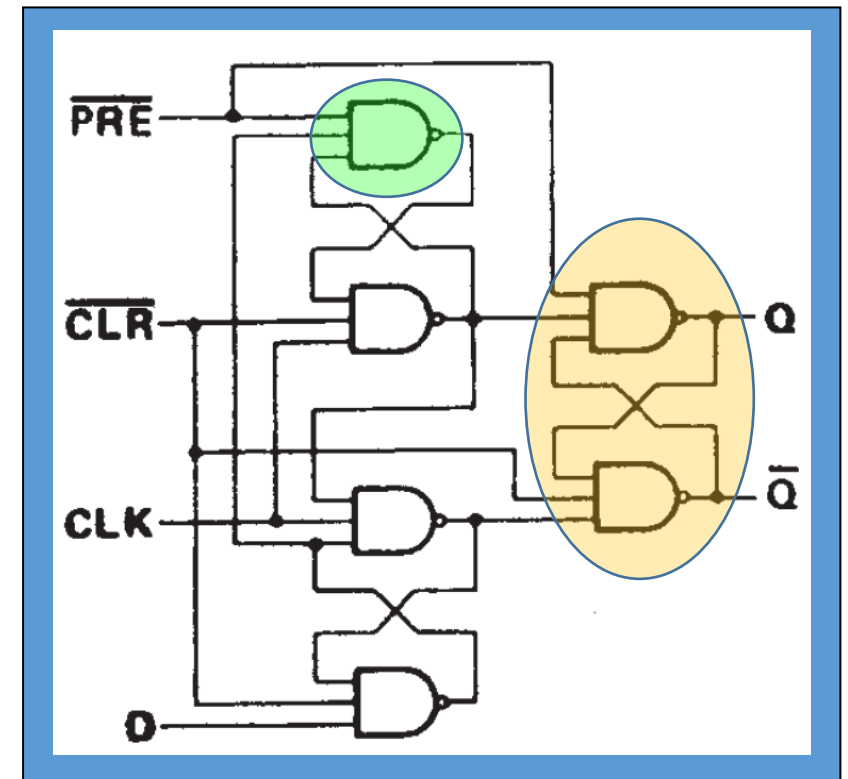
PROJETO 3 – A Primeira Implementação (**architecture**)

1. Observando o diagrama original, nota-se que o circuito usa apenas **portas NAND de 3 entradas** (digamos uma **NAND3i**)
 - Há seis portas **NAND3i** interconectadas
 - O circuito possui 4 pinos/fios de entrada, 2 pinos/fios de saída e 4 fios internos (**confira este número!!!**)
2. Com base nas observações acima crie uma descrição onde cada porta **NAND3i** é descrita em uma linha de código VHDL. Veja como se descreve uma porta lógica no projeto somador1.
 - Problema: uma NAND de 3 entradas não pode ser descrita estendendo uma NAND de 2 entradas em VHDL. Porquê? Qual a solução?
3. Necessidades
 - Declarar os fios internos
 - Os pinos de saída (Q e Q barra) são usados como entrada de portas lógicas, mas VHDL proíbe usar um pino de saída como entrada no mesmo circuito. Como resolver?

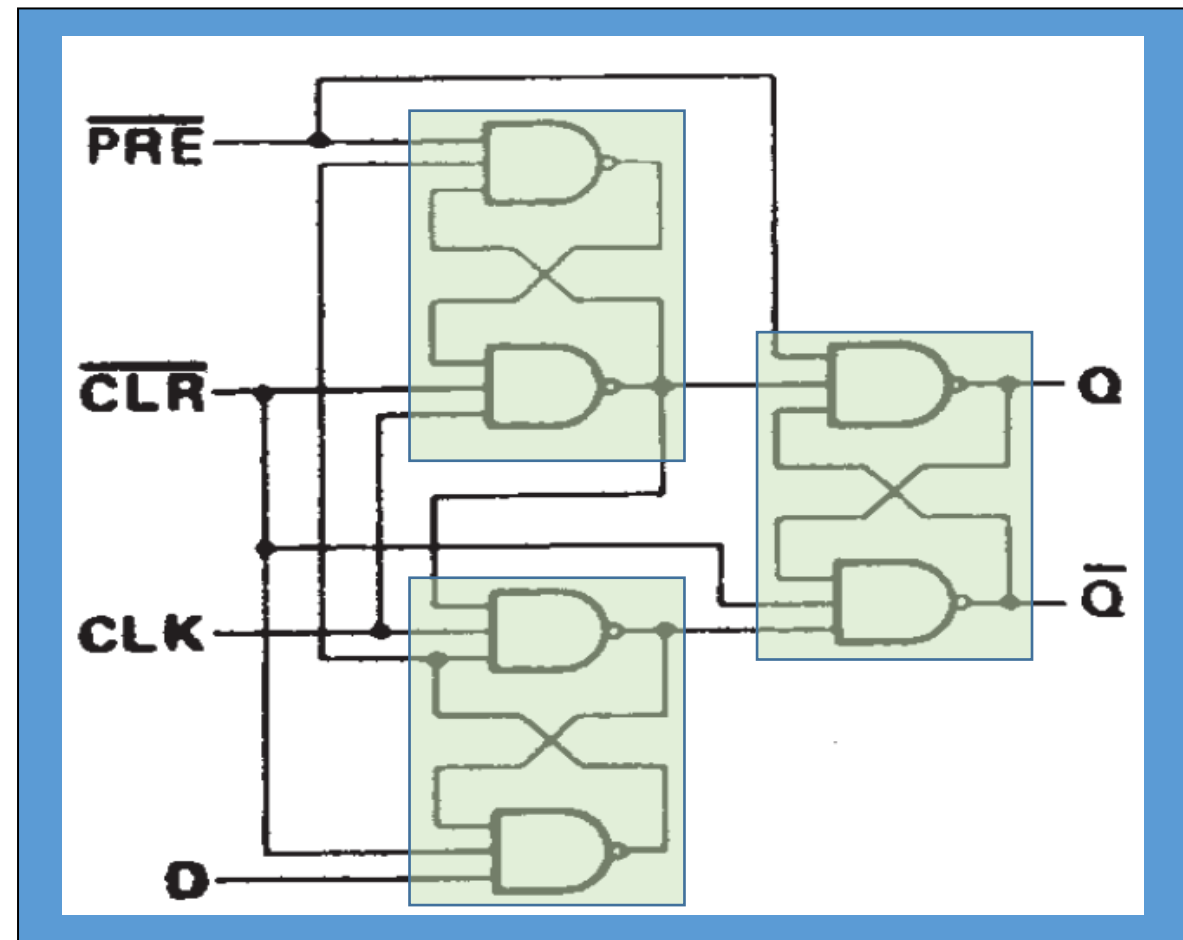
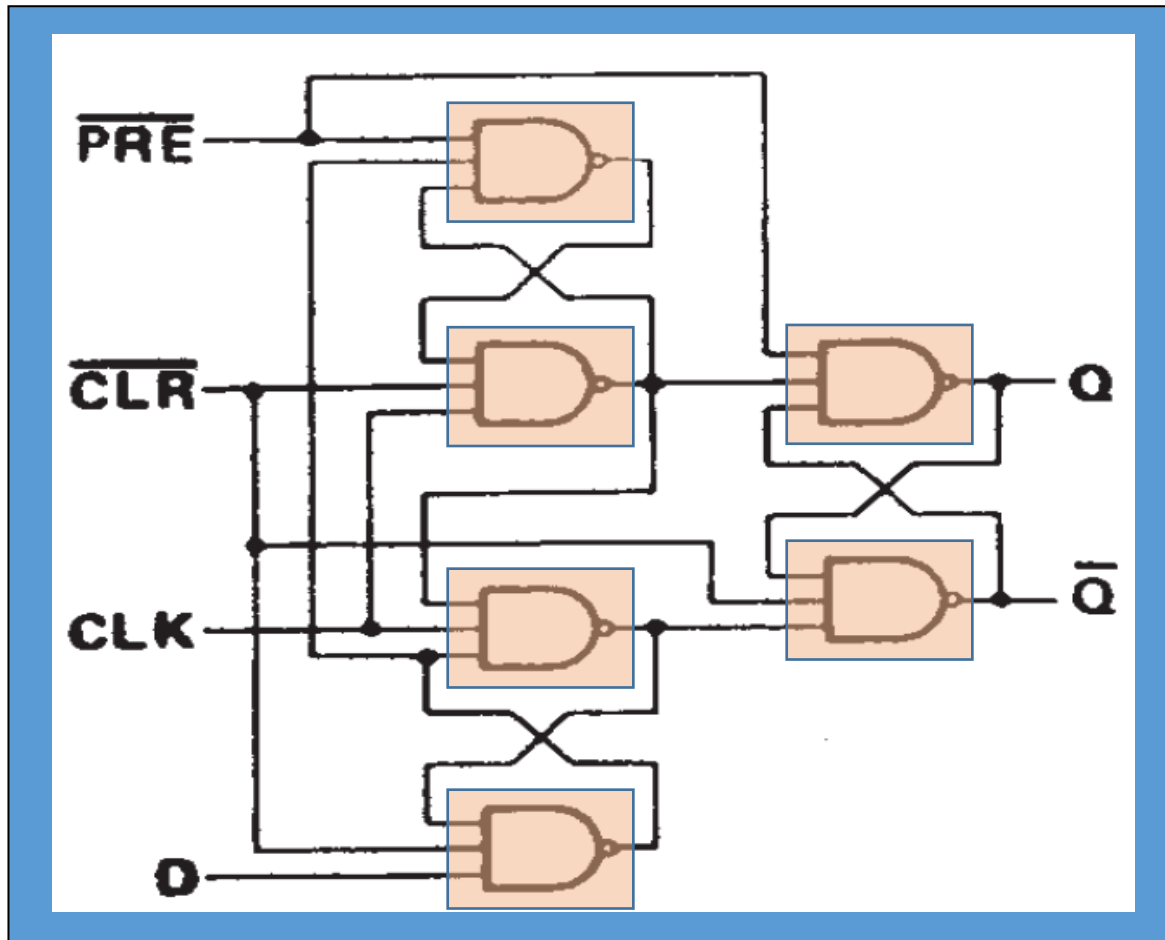


PROJETO 3 – As Implementações (**architecture**) Alternativas

1. Observando novamente o diagrama original, nota-se que existem estruturas similares repetidas no desenho
 - Como dito, o circuito só usa portas **NAND3i**
 - Há três pares de **NAND3i** interconectados do mesmo jeito (saída de uma NAND é entrada da outra e vice-versa, formando um *latch*)
2. Com base nas observações acima crie duas novas versões deste circuito em VHDL, que devem ser **hierárquicas** (??)
 - Uma usando 6 vezes uma **NAND3i** como par entidade/arquitetura
 - Uma usando 3 vezes um *latch* com **4 entradas e 2 saídas** como par entidade/arquitetura (porque 4 entradas e 2 saídas? Justifique!)
3. Estas duas novas arquiteturas lidam com descrições **modulares** e **hierárquicas** em VHDL, usando o comando **port map** de forma extensiva. Estude e aprenda como usar ele!!



PROJETO 3 – As Implementações (**architecture**) Alternativas



PROJETO 3

➤ Informações adicionais

- ✓ O circuito do Projeto 3 é um **flip-flop tipo D sensível à borda de subida do relógio** (sinal **CLK**), com **sinais assíncronos** (isto é independentes do relógio, e prioritários em relação ao sinal de relógio) de **preset** (sinal $\overline{\text{PRE}}$) e **reset** (sinal $\overline{\text{CLR}}$). Estes sinais são **ativos** (ou seja **agem**) quando valem 0. Equivale a metade de um CI TTL 7474, veja o [datasheet](#) dele
- ✓ As saídas são os sinais **Q** e $\overline{\text{Q}}$, tipicamente assumindo valores opostos, e mostram o valor do bit armazenado no **FF** e seu complemento. Use uma convenção **Q**, **Q_n** em VHDL
- ✓ O sinal $\overline{\text{PRE}}$ em 0 força a saída **Q** para 1 (e $\overline{\text{Q}}$ para 0), enquanto $\overline{\text{CLR}}$ em 0 força **Q** para 0 (e $\overline{\text{Q}}$ para 1). Eles não devem nunca ser 0 ao mesmo tempo. Qualquer um em 0 faz com que as bordas do relógio sejam ignoradas (impede o efeito deste sinal de controle)
- ✓ Seguindo a regra geral de circuitos síncronos, **nenhuma entrada deve mudar** ao mesmo tempo que o relógio. **Porquê???**
- ✓ Cada borda de subida de **CLK** sem $\overline{\text{PRE}}$ nem $\overline{\text{CLR}}$ ativados provoca a amostragem e o armazenamento do valor de **D** → funcionamento normal!!

PROJETO 3

- Notem que o circuito a implementar é sequencial e assíncrono (devido aos laços de realimentação combinacionais), e cria um componente básico para projetos síncronos. Como já foi mencionado, trata-se de um flip-flop D sensível à borda de subida do clock
- Cuidados a tomar
 - ✓ Ao escrever o *testbench*, respeitem a relação de temporização entre sinais de um circuito síncrono
 - Quando o clock mudar na sua borda sensível ($0 \rightarrow 1$, ou borda de subida), **NENHUM** sinal amostrado por ele (*D*, neste caso) pode mudar ao mesmo tempo! Lembrem-se dos conceitos de tempo de *setup* e *hold*! **Se lembram???**
 - Os sinais $\overline{\text{CLR}}$ e $\overline{\text{PRE}}$ também devem respeitar esta regra!
 - ✓ Notem, nomes de sinais no *testbench* podem ser idênticos aos nomes dos pinos dos objetos que o *testbench* instancia. Convenção: $\overline{\text{CLR}}$ pode receber nome *CLR_n*. Esta convenção pode (**deve**) ser usada para todos os sinais ativos em 0 (com barra sobre o nome)

RESUMO DO TRABALHO T1

- No Projeto 3, existem três arquiteturas (**architecture**) mas apenas uma entidade (**entity**) Estude, aprenda e use o conceito VHDL de **configuração** (**configuration**) para simular cada uma das arquiteturas que compartilham a mesma entidade
- O **Trabalho T1** compreende ao todo 3 projetos VIVADO
 - ✓ Projeto 1 - Meio-somador de 1 bit (**somador1**) (1 ponto)
 - ✓ Projeto 2 - Somador de 4 bits sem vai-um (**full_adder**) (1 ponto)
 - ✓ Projeto 3 - Flip-flop D sensível à borda do clock (**ffd**) – com três arquiteturas alternativas (6 pontos)
- O que deve ser entregue
 - ✓ Arquivo compactado (formato **ZIP** ou equivalente): **<aluno1-aluno2>.zip**, com os seguintes **arquivos**
 - ✓ Um relatório com texto, trechos de código e formas de onda, mostrando com os circuitos operam e sua estrutura (2 pontos)
 - ✓ Para cada 1 dos 3 projetos, anexe somente o código VHDL de cada implementação (três no caso do terceiro projeto) e os testbenches que validam cada implementação
 - Por exemplo **half_adder/half.vhd**, **half_adder/tb_half.vhd**, **full_adder/full.vhd**
...